



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Илија Ракочевић

**СОФТВЕРСКА АПЛИКАЦИЈА ЗА
ПРОНАЛАЖЕЊЕ ПУТАЊА У
ДВОДИМЕНЗИОНАЛНИМ ЛАВИРИНТИМА
ПРИМЕНОМ АЛГОРИТАМА ПРЕТРАГЕ**

ДИПЛОМСКИ РАД
- Основне академске студије -

Нови Сад, 2023.



КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :	
Идентификациони број, ИБР :	
Тип документације, ТД :	монографска публикација
Тип записа, ТЗ :	текстуални штампани документ
Врста рада, ВР :	дипломски (Bachelor) рад
Аутор, АУ :	Илија Ракочевић
Ментор, МН :	Владимир Иванчевић
Наслов рада, НР :	Софтверска апликација за проналажење путања у дводимензионалним лавиринтима применом алгоритама претраге
Језик публикације, ЈП :	српски
Језик извода, ЈИ :	српски
Земља публикавања, ЗП :	Србија
Уже географско подручје, УГП :	Војводина
Година, ГО :	2023
Издавач, ИЗ :	ауторски репринт
Место и адреса, МА :	Нови Сад, Факултет техничких наука, Трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страна/ цитата/табела/слика/графика/прилога)	8/51/27/0/47/0/0
Научна област, НО :	Интердисциплинарно (Електротехничко и рачунарско инжењерство; Индустриско инжењерство и инжењерски менаџмент)
Научна дисциплина, НД :	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО :	лабиринт, алгоритми претраге, визуелизација, путања
УДК	
Чува се, ЧУ :	Библиотека Факултета техничких наука, Трг Доситеја Обрадовића 6, Нови Сад
Важна напомена, ВН :	
Извод, ИЗ :	У овом раду је представљена апликација за проналажење путања коришћењем алгоритама претраге. Дат је теоријски опис коришћених алгоритама, описана је спецификација апликације и могућности су демонстриране коришћењем одабраних примера. Поред тога, дат је опис и поређење сличних апликација које су мотивисале израду овог рада.
Датум прихватања теме, ДП :	
Датум одбране, ДО :	
Чланови комисије, КО :	Председник: др Дину Драган, ванредни професор
	Члан: др Душан Гајић, ванредни професор
	Члан, ментор: др Владимир Иванчевић, ванредни професор
	Потпис ментора



KEY WORDS DOCUMENTATION

Accession number, ANO :		
Identification number, INO :		
Document type, DT :	monographic publication	
Type of record, TR :	text printed document	
Contents code, CC :	Bachelor thesis	
Author, AU :	Ilija Rakocevic	
Mentor, MN :	Vladimir Ivancevic	
Title, TI :	A software application for finding paths in two-dimensional labyrinths by using search algorithms	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian	
Country of publication, CP :	Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2023	
Publisher, PB :	author's reprint	
Publication place, PP :	Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradovića 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/51/27/0/47/0/0	
Scientific field, SF :	Interdisciplinary (Electrical and Computer Engineering; Industrial Engineering and Engineering Management)	
Scientific discipline, SD :	Applied Computer Science and Informatics	
Subject/Key words, S/KW :	labyrinth, searching algorithms, visualization, path	
UC		
Holding data, HD :	Library of the Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad	
Note, N :		
Abstract, AB :	This thesis presents an application for finding paths using search algorithms. The theoretical description of the used algorithms is given, the specification of the application is described, and the capabilities are demonstrated using selected examples. In addition, a description and comparison of similar applications that motivated the development of this thesis are provided.	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :		
Defended Board, DB :		
President:	Dinu Dragan, Ph.D., associate professor	Menthor's sign
Member:	Dušan Gajić, Ph.D., associate professor	
Member, Mentor:	Vladimir Ivančević, Ph.D., associate professor	

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Датум:
	ЗАДАТАК ЗА ИЗРАДУ ДИПЛОМСКОГ (BACHELOR) РАДА	Лист/Листова:
		1/1

(Податке уноси предметни наставник - ментор)

Врста студија:	☒ Основне академске студије
Студијски програм:	Информациони инжењеринг
Руководилац студијског програма:	др Славица Кордић, ванредни професор

Студент:	Илија Ракочевећ	Број индекса:	IN 59/2018
Област:	Интердисциплинарно (Електротехничко и рачунарско инжењерство; Индустијско инжењерство и инжењерски менаџмент)		
Ментор:	др Владимир Иванчевић, ванредни професор		

НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ДИПЛОМСКИ (Bachelor) РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:

- проблем – тема рада;
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;
- литература

НАСЛОВ ДИПЛОМСКОГ (BACHELOR) РАДА:

**СОФТВЕРСКА АПЛИКАЦИЈА ЗА ПРОНАЛАЖЕЊЕ ПУТАЊА У
ДВОДИМЕНЗИОНАЛНИМ ЛАВИРИНТИМА ПРИМЕНОМ АЛГОРИТАМА ПРЕТРАГЕ**

ТЕКСТ ЗАДАТКА:

Развити софтверску апликацију за решавање задатка проналажења путање између две назначене тачке у дводимензионалном лавиринту. Истражити могућности одабраног скупа постојећих решења која имају исту или сличну намену као апликација коју треба развити. У оквиру апликације, омогућити формирање дводимензионалног лавиринта и постављање почетне и крајње тачке путање унутар лавиринта и, за потребе проналажења саме путање, понудити могућности коришћења одабраних алгоритама претраге, како ненавођених тако и навођених. За случај навођених алгоритама претраге, у склопу апликације, омогућити употребу одабраних хеуристика заснованих на растојању. Одабране могућности апликације приказати кроз примере. Током развоја апликације, може као подршка бити искоришћена одговарајућа документација за коришћене софтверске технологије.

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора

Садржај

1. УВОД.....	6
2. СЛИЧНИ СИСТЕМИ.....	8
2.1. СОФТВЕР <i>PATHFINDING VISUALIZER</i>	8
2.2. СОФТВЕР <i>GRAPH ALGORITHMS VISUALIZER</i>	9
2.3. СОФТВЕР <i>MAZE SOLVER</i>	10
2.4. СОФТВЕР <i>PATHFINDING VISUALIZER</i>	12
2.5. ЗАКЉУЧАК О ИСТРАЖЕНИМ АПЛИКАЦИЈАМА.....	13
3. КОРИШЋЕНИ АЛГОРИТМИ.....	14
3.1. ПОМОЋНЕ СТРУКТУРЕ	14
3.2. МЕРЕЊЕ ПЕРФОРМАНСИ	14
3.3. НЕНАВОЂЕНЕ ПРЕТРАГЕ.....	15
3.3.1. ПРЕТРАГА У ШИРИНУ	15
3.3.2. УНИФОРМНА ПРЕТРАГА.....	16
3.3.3. ПРЕТРАГА У ДУБИНУ	16
3.3.4. ДВОСМЕРНА ПРЕТРАГА.....	17
3.4. НАВОЂЕНЕ ПРЕТРАГЕ.....	18
3.4.1. ПОХЛЕПНА ПРЕТРАГА.....	18
3.4.2. ПРЕТРАГА А ЗВЕЗДА	19
3.5. ФОРМИРАЊЕ ЛАВИРИНТА.....	20
3.5.1. ПРИМОВ АЛГОРИТАМ.....	20
3.5.2. РЕКУРЗИВНИ ПОВРАТАК	21
4. СПЕЦИФИКАЦИЈА АПЛИКАЦИЈЕ.....	23
4.1. ФУНКЦИОНАЛНИ ЗАХТЕВИ	23
4.2. ТЕХНОЛОГИЈЕ ИЗРАДЕ	25
5. ПРЕДСТАВЉАЊЕ АПЛИКАЦИЈЕ	27
5.1. МОТИВАЦИЈА	27
5.2. МОГУЋНОСТИ АПЛИКАЦИЈЕ.....	27
5.2.1. ЗАГЛАВЉЕ	28
5.2.2. ТЕЛО	30
5.3. ДЕМОНСТРАЦИЈА МОГУЋНОСТИ АПЛИКАЦИЈЕ КРОЗ ПРИМЕРЕ.....	36
5.3.1. ПРИМЕР 1	36
5.3.2. ПРИМЕР 2.....	40
5.3.3. ПРИМЕР 3	43
5.4. ПРЕДНОСТИ И НЕДОСТАЦИ АПЛИКАЦИЈЕ.....	47
6. ЗАКЉУЧАК.....	48
7. БИОГРАФИЈА	49
8. ЛИТЕРАТУРА	50

1. УВОД

Алгоритми претраге представљају фундаментални концепт у области рачунарских наука и вештачке интелигенције. Такође, врло су заступљени у системима заснованим на савременим технологијама као што су платформе за претрагу веба, системи за подршку одлучивању итд. Постоји више типова алгоритама претраге, али се уопштено могу класификовати у две главне категорије: ненавођене и навођене претраге. Ненавођене претраге се ослањају само на структуру проблема и не узимају у обзир додатне информације о самом доменском проблему. За разлику од ненавођених претрага, навођене претраге користе додатну информацију из доменског проблема која се назива „хеуристика”. На тај начин навођени алгоритми управљају претрагом и омогућавају многоструко боље резултате у већини случајева. Подела је укратко описна јер ће бити више речи о томе у *поглављу 3*.

Алгоритми најкраћег пута представљају специфичан тип алгоритама претраге чија је сврха проналажење најкраћег пута између почетног стања и жељеног стања. Та стања могу бити различите природе у зависности од природе проблема. Неки од познатих примера су: најкраћи пут између чворова у графу, системи глобалног позиционирања, праћење мете у видео играма итд.

Циљ овог рада јесте развој софтверске апликације која ће омогућити примену и визуелизацију алгоритама најкраћег пута у дводимензионалним лавиринтима. Како би се циљ остварио, било је потребно:

- Омогућити све потребне опције које подржавају алгоритми како би се могли симулирати.
- Осмислити изглед графичког корисничког интерфејса.
- Дефинисати начин симулације.
- Дефинисати приказ боја приликом визуелизације.

У овом раду ће бити посматрани неки од добро познатих представника из области алгоритама најкраћег пута. За ненавођене претраге су изабрани алгоритми претрага у ширину (енгл. *Breadth-First Search*), претрага у дубину (енгл. *Depth-First Search*), двосмерна претрага у ширину (енгл. *Bidirectional Breadth-First Search*), двосмерна претрага у дубину (енгл. *Bidirectional Depth-First Search*) и униформна претрага (енгл. *Uniform-Cost Search*), док су за навођене претраге изабрани похлепна претрага (енгл. *Greedy Best-First Search*) и А звезда (енгл. *A star*). Такође, за генерисање области претраге, које су лавиринти, биће коришћени Примов алгоритам (енгл. *Prim's algorithm*) и рекурзивни повратак (енгл. *Recursive Backtracker*).

У листи испод може се видети и кратак опис наредних поглавља у овом раду:

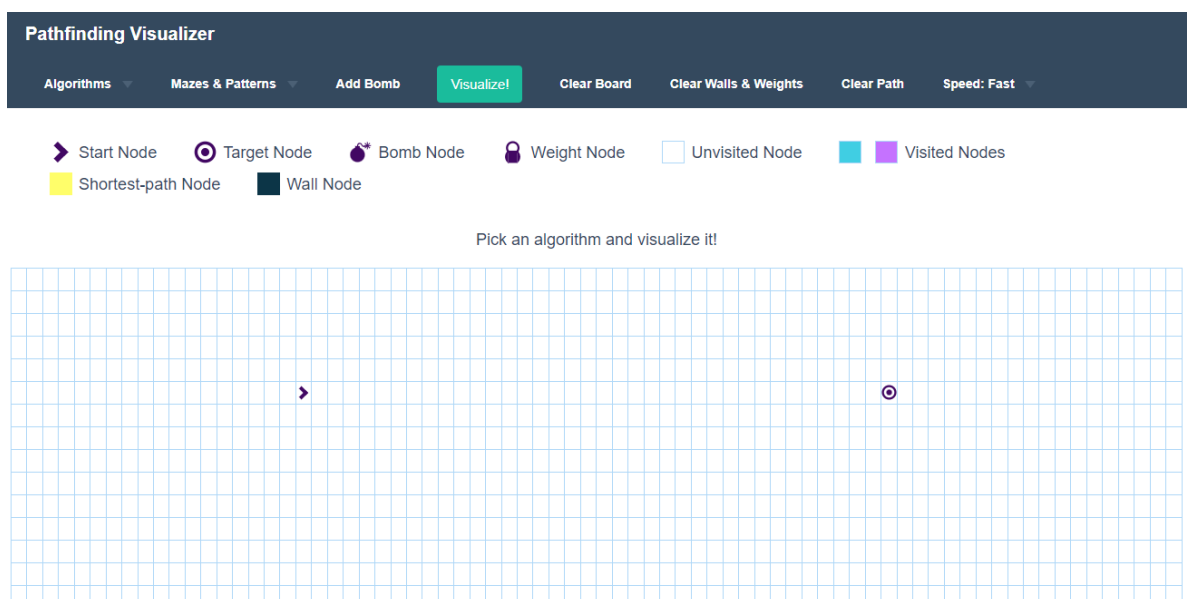
- У поглављу 2 су представљени системи који су послужили као мотивација за развој софтверске апликације описане у овом раду.
- У поглављу 3 су описани алгоритми који су имплементирани у апликацији и њихова подела.
- У поглављу 4 је представљена спецификација; како је развијена апликација и технологија у којој је имплементирана.
- У поглављу 5 су описане могућности апликације кроз примере.
- У поглављу 6 дат је закључак уз идеје за будући развој апликације.
- У поглављу 7 налази се биографија аутора овог рада.
- У поглављу 8 налази се списак литературе која је употребљена при изради овог рада.

2. СЛИЧНИ СИСТЕМИ

У овом поглављу је размотрено неколико репрезентативних софтверских решења за примену и визуелизацију алгоритама претраге. Истакнуте су следеће појединости: ко је развио софтвер, које технологије су коришћене и могућности које софтвер нуди. Могућности ће бити структуриране по категоријама како би се омогућило међусобно поређење и доношење закључака о квалитету софтвера. Уведене су следеће категорије: подржане претраге, подржани генератори лавиринта, подешавање окружења, подржан рад са тежинама чворова и подржане хеуристике.

2.1 СОФТВЕР *PATHFINDING VISUALIZER*

Софтвер под називом *Pathfinding Visualizer* објавио је Климент Михаилеску, 2016. године [1]. Технологије у којима је развијен софтвер су: *HTML*, *CSS* и *Vanilla JavaScript* [1]. На слици 2.1 се може видети изглед апликације [2].



Слика 2.1 – Изглед апликације *Pathfinding Visualizer*

Софтвер нуди следеће могућности [2]:

- Подржане претраге: Дајкстра, А звезда, похлепна претрага, рој (енгл. *Swarm*), конвергирајући рој (енгл. *Convergent Swarm*), двосмерна претрага роја (енгл. *Bidirectional Swarm*), претрага у ширину и претрага у дубину.

- Подржани генератори лавиринта: рекурзивно дељење (енгл. *Recursive Division*), рекурзивно дељење са вертикалним нагибом (енгл. *Recursive Division vertical skew*), рекурзивно дељење са хоризонталним нагибом (енгл. *Recursive Division horizontal skew*), обичан случајни лавиринт (енгл. *Basic Random Maze*), обичан лавиринт од чворова са тежином (енгл. *Basic Weight Maze*) и степенести шаблон (енгл. *Simple Stair Pattern*).
- Подешавање окружења: У овом аспектима је софтвер оскудан јер поседује само опције за брисање површине (енгл. *Clear Board*), брисање препрека и тежина (енгл. *Clear Walls & Weights*), брисање најкраћег пута (енгл. *Clear Path*) и подешавање брзине (енгл. *Speed*) која нуди три подопције: споро (енгл. *Slow*), просечно (енгл. *Average*) и брзо (енгл. *Fast*).
- Подржан рад са тежинама чворова: Омогућава избор чворова са тежином, али без могућности било каквих измена тежине.
- Подржане хеуристике: Уграђено је еуклидско растојање без могућности избора неке друге хеуристике.

2.2 СОФТВЕР GRAPH ALGORITHMS VISUALIZER

Софтвер под називом *Graph Algorithms Visualizer* објавио је Хишам Магед, 2020. године [3]. Технологије у којима је развијен софтвер су: *HTML*, *CSS* и *Vanilla JavaScript* [3]. На слици 2.2 се може видети изглед апликације [4].



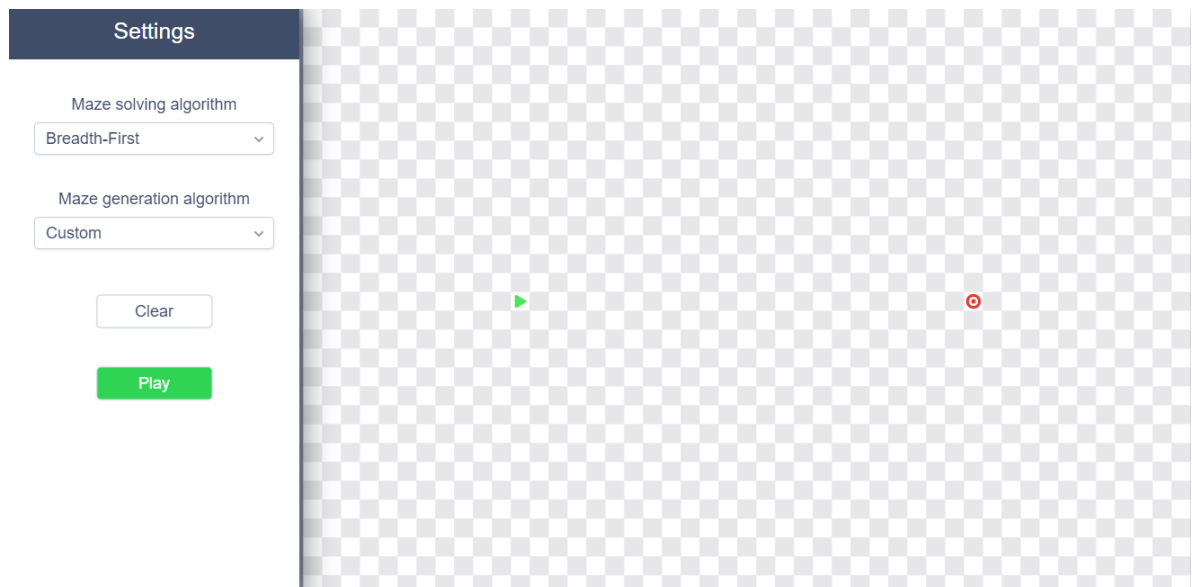
Слика 2.2 – Изглед апликације *Graph Algorithms Visualizer*

Софтвер нуди следеће могућности [4]:

- Подржане претраге: претрага у ширину, претрага у дубину, Дајкстра, двосмерна претрага у дубину, двосмерна Дајкстра претрага, А звезда са Менхетн растојањем, похлепна претрага са Менхетн растојањем, двосмерна А звезда претрага (енгл. *Bidirectional A**) и двосмерна похлепна претрага (енгл. *Bidirectional Greedy Best-First Search*).
- Подржани генератори лавиринта: рекурзивни повратак у реалном времену (енгл. *Recursive Backtracker in real time*), Примов алгоритам у реалном времену (енгл. *Prim in real time*), анимирани рекурзивни повратак (енгл. *Animated Recursive Backtracker*) и анимирани Прим (енгл. *Animated Prim*).
- Подешавање окружења: Има идентичан скуп опција за подешавање као и софтвер описан у *секцији 2.1*. То укључује брисање најкраћег пута (енгл. *Clear Path*) и брисање комплетне површине (енгл. *Clear All*) уз додатну опцију избора величине чвора (енгл. *Node Size*) на површини за визуелизацију.
- Подржан рад са тежинама чворова: Омогућава избор чворова са тежином комбинацијом тастера *Shift* + леви тастер миша, али нема могућност било каквих измена тежина.
- Подржане хеуристике: Уграђено је Менхетн растојање без могућности избора неке друге хеуристике.

2.3 СОФТВЕР MAZE SOLVER

Софтвер под називом *Maze Solver* објавио је Анђело Урју, 2021. године [5]. Технологије у којима је развијен софтвер су: *HTML*, *CSS* и *Vanilla JavaScript* [5]. На *слици 2.3* се може видети изглед апликације [6].



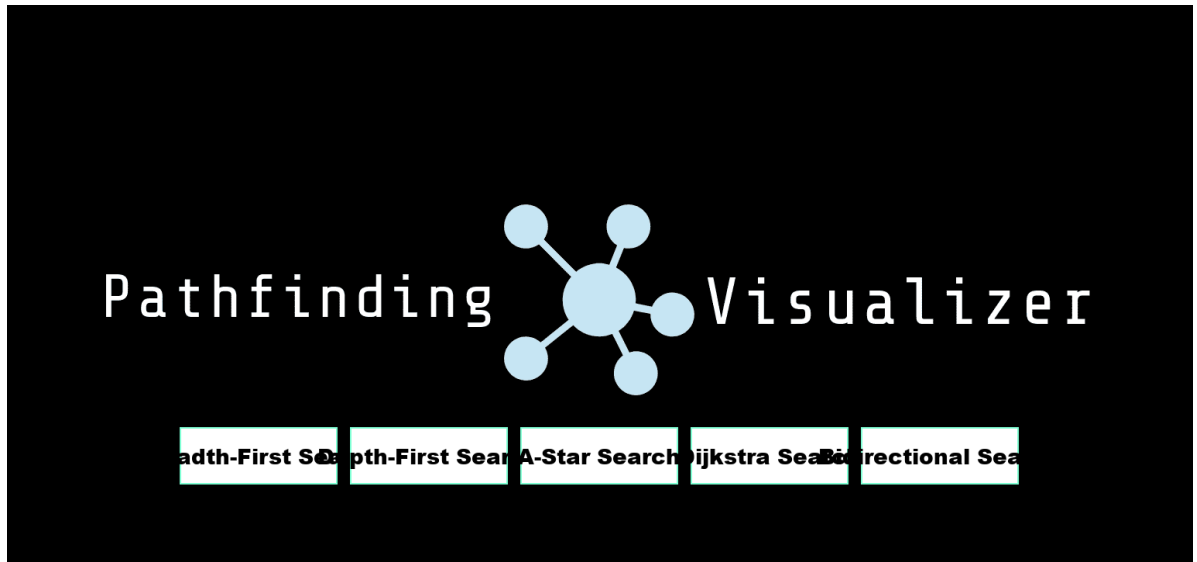
Слика 2.3 – Изглед апликације Maze solver

Софтвер нуди следеће могућности [6]:

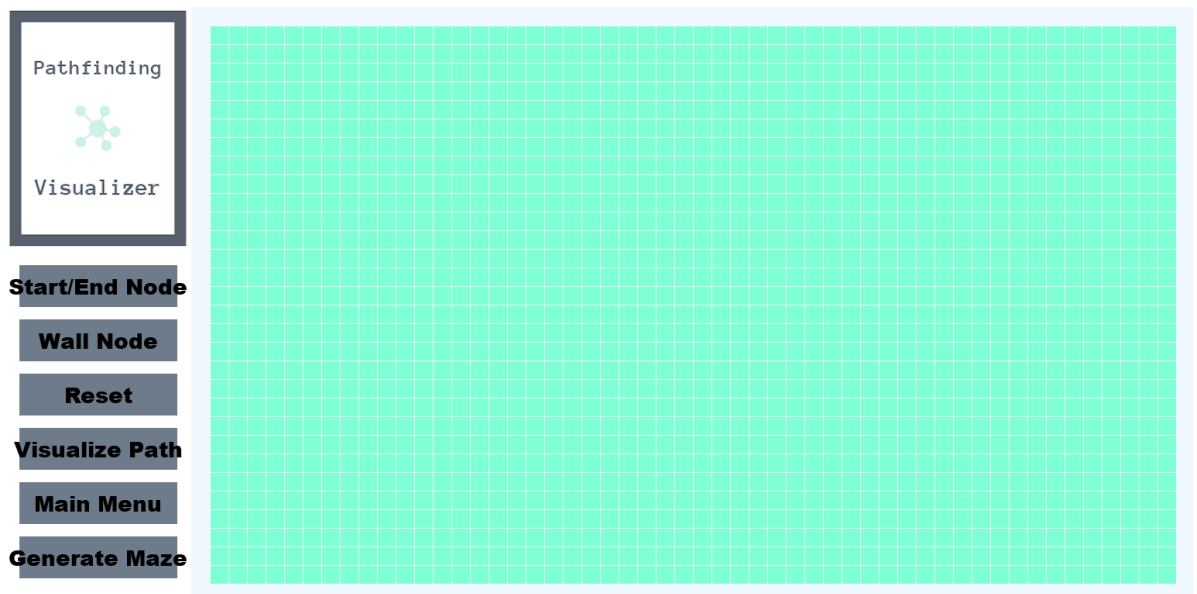
- Подржане претраге: претрага у ширину, двосмерна претрага у ширину, похлепна претрага, Дајкстра и А звезда.
- Подржани генератори лавиринта: Софтвер има обилнији избор генератора за разлику од софтвера који су представљени у *секцији 2.1* и *секцији 2.2*. Садржи следеће генераторе: случајна претрага у дубину (енгл. *Randomized Depth-First Search*), Крускалов алгоритам (енгл. *Kruskal*), Примов алгоритам (енгл. *Prim*), Вилсонов алгоритам (енгл. *Wilson*), Алдус-Бродер (енгл. *Aldous-Broder*), рекурзивно дељење (енгл. *Recursive Division*).
- Подешавање окружења: Има само једну опцију, а то је брисање површине (енгл. *Clear*).
- Подржан рад са тежинама чворова: Нема опцију за промену тежине чвора.
- Подржане хеуристике: Уграђено је еуклидско растојање без могућности избора неке друге хеуристике.

2.4 СОФТВЕР *PATHFINDING VISUALIZER*

Софтвер под називом *Pathfinding Visualizer* објавио је Сеунг Јае Ианг, 2021. Године [7]. Технологија у којој је развијен софтвер је библиотека *PyGame* за програмски језик *Python* [7]. На слици 2.4 и слици 2.5 се може видети изглед апликације [7].



Слика 2.4 – Почетни екран апликације *Pathfinding Visualizer*



Слика 2.5 – Изглед апликације *Pathfinding Visualizer*

Софтвер нуди следеће могућности [7]:

- Подржане претраге: претрага у ширину, претрага у дубину, А звезда, Дајкстра, двосмерна претрага у ширину.

- Подржани генератори лавиринта: Подржана је само рекурзивна претрага у дубину (енгл. *Recursive Depth-First Search*).
- Подешавање окружења: Нуди само опцију освежавања површине (енгл. *Reset*).
- Подржан рад са тежинама чворова: Не подржава било какав вид операција са тежинама чворова.
- Подржане хеуристике: Подржано је само еуклидско растојање.

2.5 ЗАКЉУЧАК О ИСТРАЖЕНИМ АПЛИКАЦИЈАМА

Може се приметити да све апликације имају обилан избор алгоритама за претрагу и алгоритама за формирање лавиринта. Међутим, уочен је мањак опција за подешавање окружења, немогућност или врло мала могућност утицаја на тежине чворова и немогућност избора хеуристике. Опције које постоје су углавном у вези са чишћењем простора стања у којем се претраге развијају.

Када се апликације упореде међусобно, издваја се апликација под називом *Graph Algorithms Visualizer* описана у *секцији 2.2*, која задовољава скоро све критеријуме и пружа највише слободе кориснику. Апликација која испуњава најмање задатих критеријума и даје најмање слободе кориснику је под називом *Pathfinding Visualizer* описана у *секцији 2.4*.

3. КОРИШЋЕНИ АЛГОРИТМИ

3.1 ПОМОЋНЕ СТРУКТУРЕ

Током извршавања алгоритама претраге долази до формирања стабла претраге. Сваки чвор n који припада стаблу, садржи следеће компоненте [8]:

- Стање (енгл. *State*) – стање у простору стања које одговара чвору n .
- Родитељ (енгл. *Parent*) – чвор у стаблу претраге који генерише чвор n .
- Акција (енгл. *Action*) – акција која је употребљена у родитељском чвору како би се генерисао чвор n .
- Цена пута (енгл. *Path cost*) – Традиционално се означава са $g(n)$ и представља цену пута од иницијалног чвора до чвора n .

Скуп непроширених чворова мора бити складиштен на такав начин да алгоритам може лако изабрати следећи чвор за проширење управљајући се по преферираној стратегији [8]. Одговарајућа структура података за овај случај је ред и она нуди познате операције: гурање (енгл. *push*) и скидање (енгл. *pop*). Постоје три уобичајене варијанте реда [8]:

- ФИФО (енгл. *First-in, First-out*) – враћа елемент који је први сачуван у реду.
- ЛИФО (енгл. *Last-in, First-out*) – познат и као стек (енгл. *stack*), који враћа последњи сачуван елемент у реду.
- Приоритетни (*Priority*) – враћа елемент из реда који је први у односу на задати приоритет.

3.2 МЕРЕЊЕ ПЕРФОРМАНСИ

Перформансе алгоритма се могу представити кроз четири критеријума [8]:

- Комплетност – способност да претрага увек проналази решење, под условом да решење за постављени задатак постоји.
- Оптималност – способност да претрага проналази најбоље могуће решење.
- Временска сложеност – перформансе у погледу трајања поступка претраге.

- Просторна сложеност – перформансе у погледу количине меморије потребне током извођења претраге.

3.3 НЕНАВОЂЕНЕ ПРЕТРАГЕ

3.3.1 ПРЕТРАГА У ШИРИНУ

Претрага у ширину (енгл. *Breadth-First Search*) је стратегија у којој су сви чворови на датој дубини стабла претраге проширени пре него што се пређе на наредну дубину [8]. Увођењем ФИФО реда, нови чворови, који су увек на већој дубини од својих родитеља, прелазе на крај реда, док старији чворови, који су увек на мањој дубини, први буду проширени [8]. Кораци за претрагу у ширину су следећи [9, 10]:

1. Произвољно изабрати почетни чвор и додати у ред.
2. Уколико ред није празан, уклонити чвор који је први у реду.
3. Посетити чвор који је уклоњен из реда.
4. Додати све суседе чвора који нису посећени на крај реда.
5. Понављати кораке од 2. до 4. све док се ред испразни или се пронађе циљани чвор.

Особине претраге у ширину, приликом коришћења стабла претраге су [8]:

- Комплетност – важи под условом да је фактор гранања коначан.
- Оптималност – важи ако ни за један пут који се простире из корена до дубине d' не постоји пут који се простире из корена до мање дубине d'' и има вишу цену.
- Временска сложеност – изражена преко O нотације износи $O(b^{d+1})$ или $O(b^d)$, при чему је b фактор гранања за стабло претраге, а d је дубина која одговара решењу с најмањом дубином простирања.
- Просторна сложеност – изражена преко O нотације износи $O(b^d)$, при чему је b фактор гранања за стабло претраге, а d је дубина која одговара решењу с најмањом дубином простирања.

3.3.2 УНИФОРМНА ПРЕТРАГА

За разлику од претраге у ширину, униформна претрага (енгл. *Uniform-Cost Search*) проширује чвор n са најнижом ценом пута $g(n)$ [8]. То се врши коришћењем структуре приоритетног реда, у којем је постављен приоритет по вредности g [8]. Кораци за униформну претрагу су следећи [11]:

1. Произвољно изабрати почетни чвор, доделити му цену 0 и додати у ред.
2. Уколико ред није празан, уклонити чвор са најнижом ценом из реда.
3. Посетити чвор који је уклоњен из реда.
4. Проверити да ли је посећени чвор циљани чвор:
 - 4.1 Уколико јесте циљани чвор, као резултат вратити путању.
 - 4.2 Уколико није циљани чвор, за сваког суседа посећеног чвора је потребно израчунати трошак доласка до њега. Суседа са тако израчунатим трошковима је потребно додати у ред.
5. Ако је сусед већ посећен, а нови израчунати трошак доласка до њега је нижи од претходног, потребно је ажурирати цену и путању до суседа.
6. Понављати кораке од 2. до 5. све док се ред испразни или се пронађе циљани чвор.

Особине униформне претраге, приликом коришћења стабла претраге су [8]:

- Комплетност – важи под условом да је фактор гранања коначан и да су цене корака веће или једнаке с неком позитивном константом ε .
- Оптималност – важи под условом да не постоје негативне цене корака.
- Временска сложеност – изражена преко O нотације износи $O(b^{1+\lceil C^*/\varepsilon \rceil})$, при чему је b фактор гранања за стабло претраге, C^* је цена оптималног решења, а ε је претпостављена најнижа позитивна цена корака.
- Просторна сложеност – изражена преко O нотације износи $O(b^{1+\lceil C^*/\varepsilon \rceil})$, при чему је b фактор гранања за стабло претраге, C^* је цена оптималног решења, а ε је претпостављена најнижа позитивна цена корака.

3.3.3 ПРЕТРАГА У ДУБИНУ

Претрага у дубину (енгл. *Depth-First Search*), код стабла претраге, је стратегија у којој се посећивањем чворова понире до што дубљег нивоа стабла; уколико нема наредног чвора за посећивање, алгоритам се враћа на последњи посећен чвор и узимајући непосећеног потомка тог чвора наставља понирање. Претрага у дубину се може имплементирати на два начина; преко рекурзије

или коришћењем ЛИФО реда [8]. Кораци за претрагу у дубину су следећи [12, 13]:

1. Произвољно изабрати почетни чвор и додати у ред.
2. Уколико ред није празан, уклонити чвор који је последњи додат у ред.
3. Посетити чвор који је уклоњен из реда.
4. Додати све суседе чвора који нису посећени у ред.
5. Понављати кораке од 2. до 4. све док се ред не испразни.

Особине претраге у дубину, приликом коришћења стабла претраге су [8]:

- Комплетност – не важи.
- Оптималност – не важи.
- Временска сложеност – изражена преко O нотације износи $O(b^m)$, при чему је b фактор гранања за стабло претраге, а m је максимална дубина у стаблу претраге.
- Просторна сложеност – изражена преко O нотације износи $O(bm)$, при чему је b фактор гранања за стабло претраге, а m је максимална дубина у стаблу претраге.

3.3.4 ДВОСМЕРНА ПРЕТРАГА

Идеја иза двосмерног претраживања (енгл. *Bidirectional Search*) је покретање две истовремене претраге – једна од почетног стања, а друга од циљаног стања – са циљем да се две претраге сретну на средини [8]. Двосмерно претраживање се имплементира на начин да се више не спроводи тест за циљано стање, већ се проверава да ли се две претраге укрштају; ако се укрсте, решење је пронађено [8]. Кораци за двосмерну претрагу су следећи [14]:

1. Изабрати почетни и циљани чвор.
2. Иницијализовати два реда: један за претрагу унапред, а други за претрагу уназад.
3. Додати почетни чвор у ред за претрагу унапред, а циљани чвор у ред за претрагу уназад.
4. Уколико није празан ниједан од редова, урадити следеће кораке:
 - 4.1 Уклонити чвор из реда за претрагу унапред и означити га као посећеног.
 - 4.2 Сваки непосећени суседни чвор од последњег уклоњеног чвора додати у ред за претрагу унапред и означити га као посећеног.

- 4.2.1 Ако се током претраге унапред пронађе посећени чвор који је већ посећен током претраге уназад, пронађена је најкраћа путања.
- 4.3 Уклонити чвор из реда за претрагу уназад и означити га као посећеног.
- 4.4 Сваки непосећени суседни чвор од последњег уклоњеног чвора додати у ред за претрагу уназад и означити га као посећеног.
- 4.4.1 Ако се током претраге уназад пронађе посећени чвор који је већ посећен током претраге унапред, пронађена је најкраћа путања.
- 5. Вратити најкраћи пут између изворног чвора и циљаног чвора.

Особине претраге у дубину, приликом коришћења стабла претраге су [8, 15]:

- Комплетност – може важити под условом да је фактор гранања b коначан, али зависи и од врсте претрага које се користе. На пример, комплетност важи уколико је претрага у ширину и као претрага унапред и као претрага уназад.
- Оптималност – зависи од врсте претраге која се користи и ценовне функције. На пример, оптималност важи уколико је претрага у ширину и као претрага унапред и као претрага уназад. Такође важи, ако ни за један пут који се простире из корена до дубине d' не постоји пут који се простире из корена до мање дубине d'' и има вишу цену.
- Временска сложеност – изражена преко O нотације износи $O(b^{d/2})$, при чему је b фактор гранања за стабло претраге, а d је дубина која одговара решењу с најмањом дубином простирања.
- Просторна сложеност – изражена преко O нотације износи $O(b^{d/2})$, при чему је b фактор гранања за стабло претраге, а d је дубина која одговара решењу с најмањом дубином простирања.

3.4 НАВОЂЕНЕ ПРЕТРАГЕ

3.4.1 ПОХЛЕПНА ПРЕТРАГА

Похлепна претрага (енгл. *Greedy Best-First Search*) је стратегија, која приликом претраге простора стања, даје приоритет чвору n са најнижом вредношћу хеуристичке функције $h(n)$ без икаквог разматрања трошкова доласка до тог чвора [8]. Функција процене је дата преко формуле [17]:

$$f(n) = h(n) \quad (1)$$

Из формуле се може приметити да алгоритам узима у обзир само вредност хеуристичке функције и она је једини фактор при одлуци о приступању наредном чвору. Иако је приступ ефикасан у пракси, врло је подложен грешкама уколико није одабрана погодна хеуристика [18]. Имплементација се врши коришћењем структуре приоритетног реда [8]. Кораци за похлепну претрагу су следећи [19]:

1. Произвољно изабрати почетни чвор и додати у ред.
2. Уколико ред није празан, уклонити чвор са најнижом вредношћу хеуристичке функције из реда.
3. Посетити чвор који је уклоњен из реда.
4. Проверити да ли је посећени чвор циљани чвор
 - 4.1 Уколико јесте циљни чвор, зауставити претрагу и вратити путању од почетног чвора до циљаног чвора.
 - 4.2 Уколико није циљани чвор, ставити у ред све непосећене суседне чворове уклоњеног чвора и означити их као посећене.
5. Понављати кораке од 2. до 4. све док се ред испразни или се пронађе циљани чвор.

Особине похлепне претраге, приликом коришћења стабла претраге су [8]:

- Комплетност – не важи.
- Оптималност – не важи.
- Временска сложеност – изражена преко O нотације износи $O(b^m)$, при чему је b фактор гранања за стабло претраге, а m је максимална дубина.
- Просторна сложеност – изражена преко O нотације износи $O(b^m)$, при чему је b фактор гранања за стабло претраге, а m је максимална дубина.

3.4.2 ПРЕТРАГА А ЗВЕЗДА

Најпознатија форма претраге по најбољем првом кораку (енгл. *Best-First Search*) назива се А звезда (енгл. *A* Search*), која процењује чворове комбиновањем $g(n)$ и $h(n)$, при чему $g(n)$ представља цену за приступ чвору, док $h(n)$ представља укупну цену од чвора n до циљаног чвора [8]. Имплементација се врши коришћењем структуре приоритетног реда, док формула по којој се израчунава процена је [8]:

$$f(n) = g(n) + h(n) \quad (2)$$

Кораци за А звезда претрагу су следећи [20]:

1. Произвољно изабрати почетни чвор и додати у ред.
2. Уколико ред није празан, уклонити чвор са најнижим укупним трошковима, који се добијају помоћу *формуле 2*.
3. Посетити чвор који је уклоњен из реда.
4. Проверити да ли је посећени чвор циљани чвор.
 - 4.1 Уколико јесте циљани чвор, зауставити претрагу и вратити путању од почетног чвора до циљаног чвора.
 - 4.2 Уколико није циљани чвор, ставити у ред све непосећене суседне чворове уклоњеног чвора и израчунати њихову укупну цену.
 - 4.2.1 Уколико је нова укупна цена суседног чвора мања од његове старе укупне цене, ажурира се цена и родитељски чвор у реду.
5. Понављати кораке од 2. до 4. све док се ред испразни или се пронађе циљани чвор.

Особине А звезда претраге, приликом коришћења стабла претраге су [8]:

- Комплетност – важи под условом да је фактор гранања b коначан и да су цене корака веће или једнаке са неком позитивном константом ε .
- Оптималност – важи под условом да је фактор гранања b коначан и све цене корака су веће или једнаке с неком позитивном константом ε и коришћена хеуристичка функција $h(n)$ је допустива хеуристика.
- Временска сложеност – зависи од одабране хеуристике и природе задатка. Може бити експоненцијалног карактера у одређеним случајевима, при чему је основа фактор гранања за стабло претраге означен са b , док се експонент односи на грешку хеуристике (одступање коришћене хеуристике од идеалне хеуристике).
- Просторна сложеност – потребно је чувати податке о свим формираним чворовима.

3.5 ФОРМИРАЊЕ ЛАВИРИНТА

3.5.1 ПРИМОВ АЛГОРИТАМ

У рачунарству, Примов алгоритам (енгл. *Prim's algorithm*), је алгоритам похлепе који налази минимално простируће стабло за тежински неусмерен граф [21]. Алгоритам ради тако што гради стабло чвор по чвор, почевши од

произвољног почетног чвора, при сваком кораку додајући најјефтинију могућу везу из стабла ка другом чвору [21]. Кораци за Примов алгоритам су следећи [21]:

1. Произвољно изабрати почетни чвор из графа.
2. Иницијализација стабла са изабраним чвором.
3. Ширење стабла избором једне гране: од грана које повезују стабло са чворовима који нису у стаблу, пронаћи грану минималне тежине и додати је у стабло.
4. Понављати трећи корак док се сви чворови не нађу у стаблу.

Особине Примовог алгоритма су [20]:

- Комплетност – увек ће важити уколико је граф повезан, тежински и неусмерен.
- Оптималност – увек ће важити уколико је граф повезан, тежински и неусмерен.
- Временска сложеност – изражена преко O нотације износи $O(E \log V)$, при чему је V број чворова графа, а E је број грана графа.
- Просторна сложеност – изражена преко O нотације износи $O(V + E)$, при чему је V број чворова графа, а E је број грана графа.

3.5.2 РЕКУРЗИВНИ ПОВРАТАК

Алгоритам рекурзивног повратка (енгл. *Recursive Backtracker*) је добро познати алгоритам за генерисање лавиринта који користи претрагу у дубину како би креирао исти [22]. За разлику од претраге у дубину, алгоритам рекурзивног повратка не користи ЛИФО ред за праћење посећених чворова, већ прати тренутни пут рекурзивним позивима самог себе све док не стигне до ћорсокака [22].

Може се имплементирати користећи различите структуре података, као што су дводимензионални низ за приказ простора стања, стек за праћење тренутног пута и генератора случајних бројева за избор следеће ћелије за посету [22]. Алгоритам је једноставан и ефикасан и често се користи у играма, симулацијама и другим апликацијама где је то потребно [22].

Алгоритам се може неформално описати следећим корацима [22]:

1. Произвољно изабрати почетни чвор и означити га као посећеног.
2. Уколико у простору стања постоје чворови које нису посећени:
 - 2.1 Приступити насумичном непосећеном суседу тренутно изабраног чвора.

- 2.1.1 Уколико нема непосећених чворова, вратити се на најскорији чвор који има непосећене суседне чворове.
 - 2.2 Изабрани суседни чвор означити као посећен и формирати везу са тренутним чвором.
 - 2.3 Изабрани суседни чвор поставити као нови тренутни чвор.
- 3. Када су сви чворови посећени, лавиринт је формиран.

4. СПЕЦИФИКАЦИЈА АПЛИКАЦИЈЕ

4.1 ФУНКЦИОНАЛНИ ЗАХТЕВИ

Као први корак, требало је препознати које захтеве апликација мора да испуни. Како би сви потребни аспекти ове области били што боље обухваћени и то са што мање пропуста, извршена је детаљна анализа помоћних структура алгоритама, хеуристика које ће бити остављене на избор кориснику, структура података које се користе у алгоритмима итд.

Из *поглавља 3*, које теоријски описује метод рада алгоритама, препознати су одређени захтеви чија се имплементација мора обезбедити у апликацији. Препознати захтеви су: дефинисање простор стања у којем ће се развијати претраге, дефинисање чвора као основног елемента простора стања, дефинисање скупа стања у којима може бити чвор, могућност бирања позиције почетног и циљаног чвора, могућност избора хеуристике, рачунање и приказивање најкраћег пута и структура података која ће чувати чворове током развоја претраге.

Простор стања – закључено је да ће простор стања у апликацији бити представљен помоћу дводимензионалне површине базиране на изгледу решетке, при чему се димензије површине могу мењати.

Чвор – у имплементацији представља основни елемент поменутог простора стања и представљен је помоћу квадрата.

Стање чвора – током развоја претраге, долази до константне промене стања чворова. Један од примера је прелазак из стања „непосећен” у стање „посећен”. Стања препозната из теоријског описа алгоритама су изражена у апликацији помоћу следећег дефинисаног скупа: почетни чвор, циљани чвор, препрека, непосећени чвор са тежином једнаком јединици, непосећени чвор са подешеном тежином, посећени чвор са тежином једнаком јединици, посећени чвор са подешеном тежином, чвор са тежином једнаком јединици који је део најкраћег пута и чвор који је део најкраћег пута и има подешену тежину. Свако стање има другачији визуелни приказ и сваки чвор може прећи у било које стање. Визуелни приказ скупа стања чворова се може видети на *слици 4.1*. Опис свих стања:

- Почетни чвор (енгл. *Start Node*) и циљани чвор (енгл. *Goal Node*) – Детаљи о додели почетног и крајњег чвора су приказани у *секцији 5.2.2*. Почетно стање је представљено тамно плавом бојом, док је циљано стање представљено тамно црвеном бојом.
- Препрека (енгл. *Wall Node*) – чвор који је у стању препреке се не разматра приликом развоја претраге и служи као граница. Уколико се развојем претраге сусретне ово стање, морају се тражити алтернативни чворови. Стање препреке је представљено помоћу тегет боје.

- Непосећени чвор са тежином једнаком јединици (енгл. *Unvisited Node*) – у овом стању су сви они чворови до којих развој претраге још није стигао, а истовремено немају подешену тежину од стране корисника. Стање је визуелно представљено помоћу празног квадрата са тегет ивицама.
- Непосећени чвор са подешеном тежином (енгл. *Unvisited Weighted Node*) – стање које означава да чвор има тежину већу од један. Вредност тежине која може бити додељена овом стању је у интервалу од 2 до 20, укључујући и граничне вредности. Кориснику је омогућена промена ових вредности. Чвор са подешеном тежином разматрају само претраге које покушавају да минимизују цену пута до циљаног чвора. Остале претраге ће сматрати да је тежина чвора са овим стањем једнака јединици. Пример сценарија је приказан у *секцији 5.3.3*. Стање је визуелно представљено разним нијансама зелене боје, при чему нијансе зависе од тежине чвора.
- Посећени чвор са тежином једнаком јединици (енгл. *Visited Node*) – стање у које прелазе непосећени чворови чија је тежина била једнака јединици. Стање је представљено помоћу две боје; светле плаве боје и светле црвене боје. Светла црвена боја се користи код двосмерних претрага за приказ развоја претраге уназад, док је светла плава заступљена код свих осталих претрага.
- Посећени чвор са подешеном тежином (енгл. *Visited Weighted Node*) – стање у које прелази чвор који има подешену тежину и узет је у обзир током развоја претраге. Визуелно је представљен плавом бојом са подебљаним ивицама које су тегет боје.
- Чвор са тежином једнаком јединици који је део најкраћег пута (енгл. *Path Node*) – у ово стање прелазе чворови који су посећени, чија је тежина једнака јединици и за које је алгоритам утврдио да припадају најкраћем путу. Стање је визуелно приказано помоћу светло жуте боје.
- Чвор са подешеном тежином који је део најкраћег пута (енгл. *Weighted Path Node*) – у ово стање прелазе чворови који су посећени, чија је тежина подешена од стране корисника и за које је алгоритам утврдио да припадају најкраћем путу. Стање је визуелно приказано помоћу жуте боје са подебљаним ивицама које су тегет боје.

Произвољно бирање почетног и циљаног чвора – развој претраге почиње од произвољног почетног чвора и завршава се доласком до произвољног циљаног чвора. Из тог разлога је кориснику обезбеђен механизам бирања позиције почетног и циљаног чвора, који је објашњен у *секцији 5.2.2*.

Хеуристика – навођене претраге захтевају хеуристику како би функционисале. У апликацији је имплементирано неколико врста хеуристика

које корисник може бирати. Детаљи око коришћења хеуристика су објашњени у *секцији 5.2.1*, а њихов утицај на претраге је приказан у *секцији 5.3.2*.

Најкраћи пут – крајњи резултат успешне претраге је најкраћи пут од почетног до циљаног стања. Апликација испуњава овај захтев тако што на крају сваке претраге прикаже најкраћи пут посебно дефинисаном бојом.

Структура података – потребно је обезбедити структуру података у којој ће се чувати чворови током развоја претраге. *JavaScript* програмски језик, чији опис је дат у *секцији 4.2*, подржава рад са структуром података ред (енгл. *queue*), који је у претрази потребан.

Поред тога, корисник може да уређује дводимензионалну површину како он жели. На пример, избором препреке као стања чвора, може означити све чворове које жели да буду препрека у претрази и на тај начин формирати терен који је погодан за истраживање. Окружење имплементирано на овај начин, омогућава кориснику да симулира рад претрага, а сама апликација подржава све што је претрагама потребно како би функционисале.



Слика 4.1 – Визуелни приказ могућих стања чвора

4.2 ТЕХНОЛОГИЈЕ ИЗРАДЕ

Изабран је веб као тип апликације и програмски језик у којем ће се имплементирати апликација јесте *JavaScript*. Конкретно ће се користити структура *DOM (Document Object Model)* стабла за приступ и манипулацију елементима, коју пружа поменути језик. *HTML* и *CSS* ће бити коришћени за уређивање изгледа апликације.

HTML (Hypertext Markup Language) је језик за означавање који се користи за креирање и структурирање садржаја на веб страницама [23]. Први пут је представљен 1993. године и од тада је постао стандардни језик који се користи за креирање веб страница и апликација [24]. Састоји се од низа ознака које дефинишу различите елементе на веб страници, као што су наслови, пасуси, слике, везе и обрасци [23].

CSS (Cascading Style Sheets) је језик који се користи за описивање презентације и изгледа веб страница [25]. Први пут је представљен 1996. године и од тада је постао основна технологија која се користи у развоју веба [25]. *CSS* се користи за дефинисање визуелних аспеката веб странице, као што су боје, фонтови, изглед и размаци [25]. Омогућава програмерима да одвоје презентацију веб странице од њеног садржаја, што олакшава одржавање и ажурирање веб страница [25].

JavaScript је програмски језик високог нивоа, динамичан и интерпретиран који се широко користи за креирање интерактивних веб страница и апликација [26]. Брендан Ајк га је 1995. године увео у употребу и од тада је постао један од најпопуларнијих програмских језика на свету [27]. Према статистици, искоришћен је за прављење 95,2% веб сајтова у 2019. години [27].

5. ПРЕДСТАВЉАЊЕ АПЛИКАЦИЈЕ

5.1 МОТИВАЦИЈА

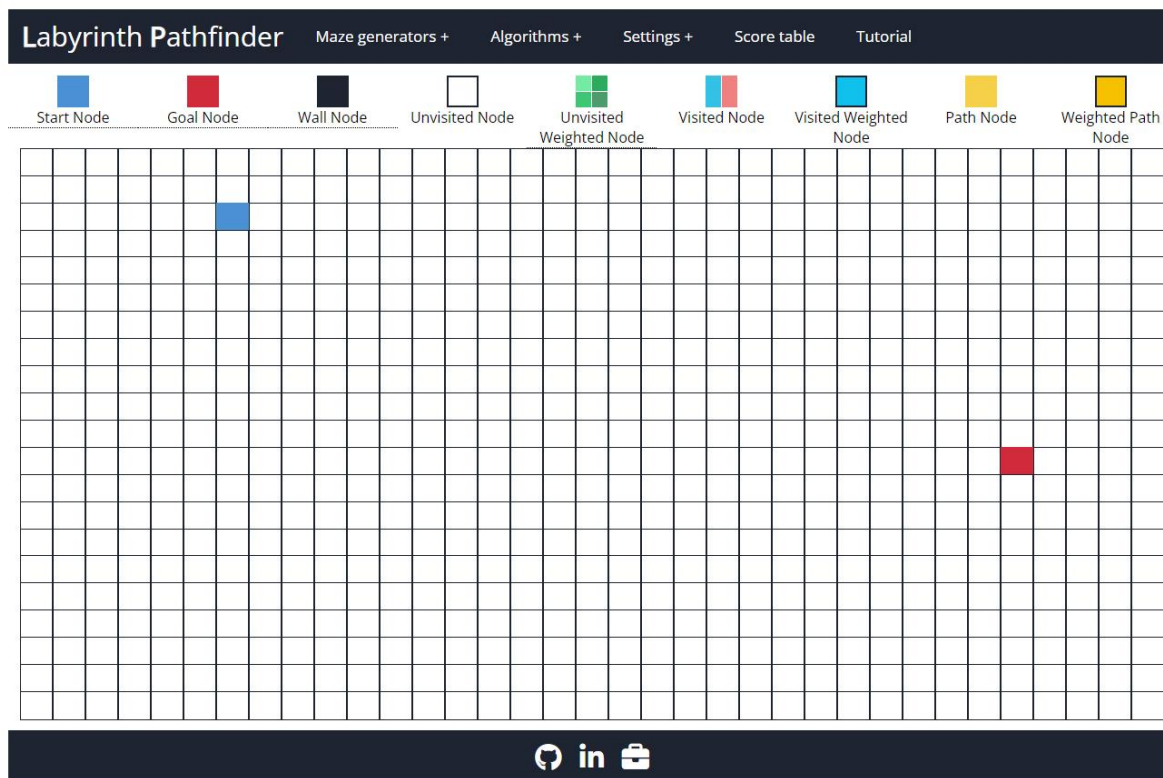
Потребно је истакнути да је као мотивација за развој ове апликације послужила првенствено апликација описана у *секцији 2.1* од аутора Климента Михаилескуа [1, 2]. Михаилеску је међу првима представио овакав тип визуелизације алгоритама најкраћег пута. Апликације за визуелизацију су биле присутне и пре његовог пројекта, али ни у једној није омогућена слобода кориснику да мења окружење и посматра развој алгоритама у реалном времену. Касније су се појавиле и многе друге апликације са сличним могућностима.

5.2 МОГУЋНОСТИ АПЛИКАЦИЈЕ

У апликацији се користе речи преведене на енглески језик. Разлог превода је могућност употребе стандардизованих и устаљених термина из ове области који су коришћени у већини литературе и који су добро познати. Као други разлог може се навести пружање могућности да се апликација користи од стране већег броја корисника.

Графички кориснички интерфејс је подељен у три секције, које се могу видети на *слици 5.1*:

- Заглавље (енгл. *Header*) – садржи опције за рад са апликацијом и приказ могућих стања чворова;
- Тело (енгл. *Body*) – садржи простор стања преко којег ће се вршити визуелизација алгоритама претраге;
- Подножје (енгл. *Footer*) – садржи линкове ка *GitHub* и *LinkedIn* профилима и портфолију.



Слика 5.1 – Изглед апликације

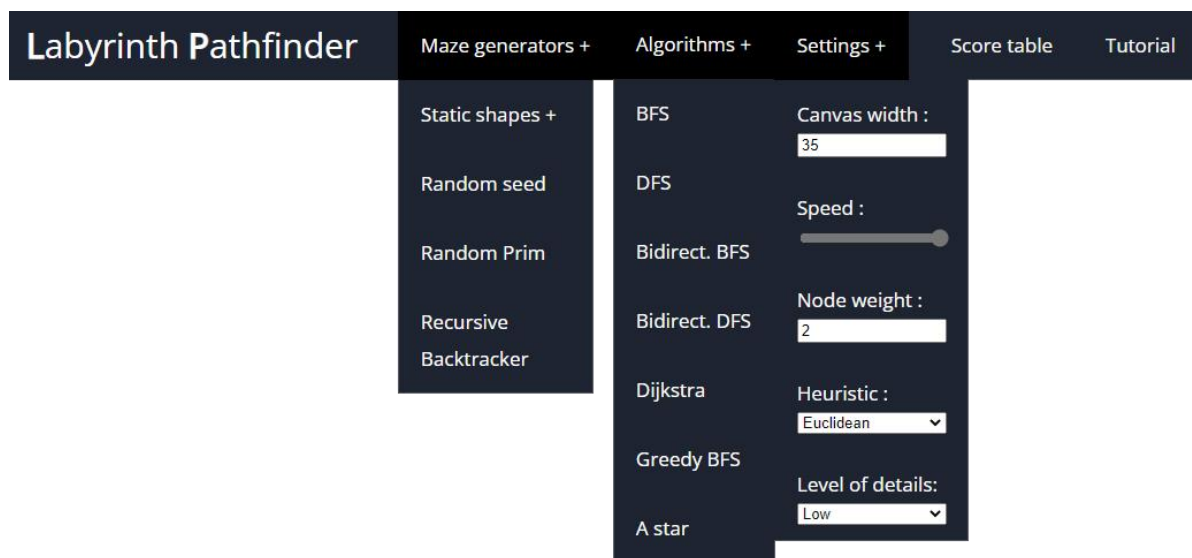
5.2.1 ЗАГЛАВЉЕ

Опције које нуди заглавље су:

- Генератори лавиринта (енгл. *Maze generators*) – корисник може изабрати генератор и на тај начин променити простор стања у којем ће се развијати алгоритам. Могућ је избор случајног семена (енгл. *Random seed*), случајног Прим алгоритма (енгл. *Randomized Prim*), рекурзивног повратка (енгл. *Recursive Backtracker*) и статичких лавирината (енгл. *Static shapes*) који су увек исти. Подржани статички лавиринти су: степенasti лавиринт (енгл. *Stairs maze*), централни лавиринт (енгл. *Central maze*), хоризонтални лавиринт (енгл. *Horizontal maze*), вертикални лавиринт (енгл. *Vertical maze*) и нормални лавиринт (енгл. *Normal maze*). Приказ листе понуђених генератора се може видети на слици 5.2.
- Алгоритми (енгл. *Algorithms*) – корисник може изабрати алгоритам претраге који жели. Приказ листе алгоритама се може видети на слици 5.2.
- Подешавања (енгл. *Settings*) – омогућавају промену величине простора стања, промену брзине визуелизације, промену тежине чвора, избор хеуристике за навођене претраге и избор нивоа детаља који ће се

приказивати у простору стања. Подржане хеуристике су: еуклидска (енгл. *euclidean*), Менхетн (енгл. *Manhattan*), Хемингова (енгл. *Hamming*), хеуристика Минковског (енгл. *Minkowski*), косинусна (енгл. *Cosine*), Чебишевљева (енгл. *Chebyshev*) и хеуристика хаверсинуса (енгл. *Haversine*). Режим високог нивоа детаља (енгл. *High*) приказује додатне информације о чворовима у простору стања. Додатне информације су: тежина коју има чвор, визуелизација чвора прилагођена његовој тежини и уколико постоје чворови са тежином који су посећени током претраге, они су посебно назначени у простору стања. Приказ подешавања се може видети на *слици 5.2*.

- Табела резултата (енгл. *Score table*) – бележи основне податке о свакој претрази која је извршена од последњег покретања апликације. Табела има опције: брисање појединачних редова у табели (енгл. *Clear*), преузимање фотографије у *png* формату (енгл. *Download ss*) која приказује крајњи сценарио претраге која је изабрана, брисање свих резултата (енгл. *Clear all*) и преузимање табеле у *csv* формату (енгл. *Download CSV*), који може послужити за касније анализе. Изглед табеле резултата се може видети на *слици 5.3*.
- Инструкције (енгл. *Tutorial*) – последња опција у заглављу описује на који начин се користи апликација.



Слика 5.2 – Приказ избора генератора лавиринта, избора алгоритма и подешавања

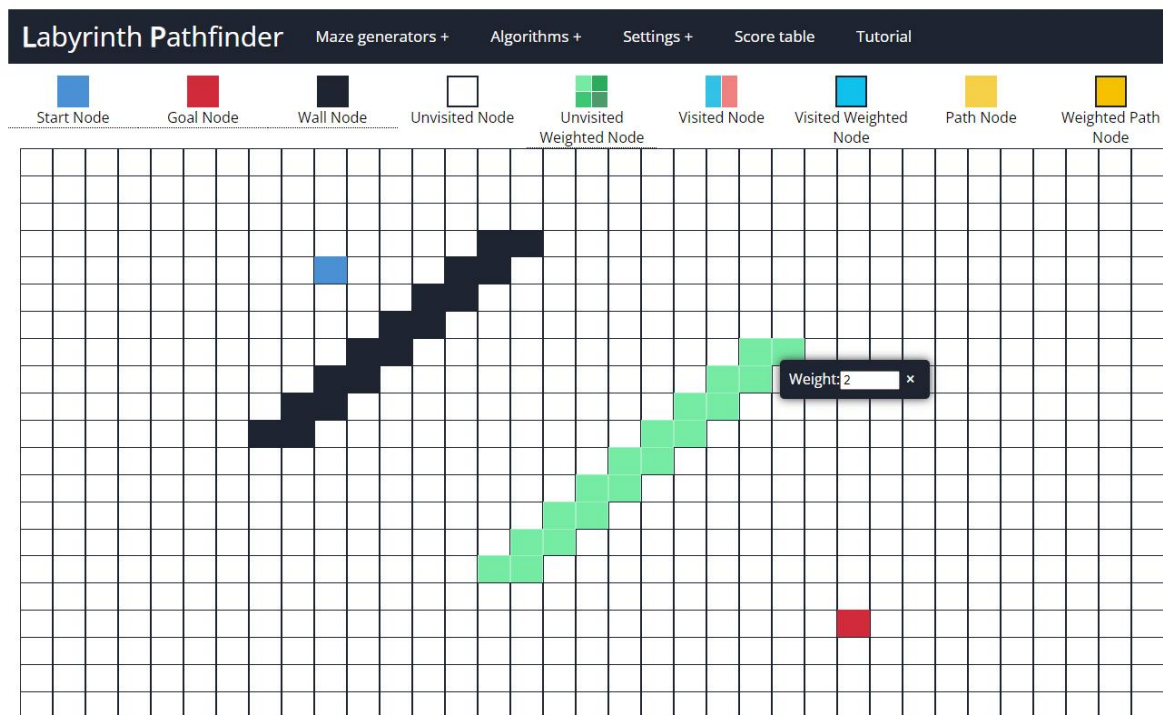
Score Table									
Alg. name	Path length	Visited nodes	Time	Canvas	Speed	Heuristic	Details	Operations	
BFS	28	620	3483.00	35	500	/	Low	Clear	Download ss
DFS	384	384	2134.90	35	500	/	Low	Clear	Download ss
Bidirect. BFS	28	533	1619.20	35	500	/	Low	Clear	Download ss
Bidirect. DFS	334	569	2079.80	35	500	/	Low	Clear	Download ss
Dijkstra	28	622	3495.60	35	500	/	Low	Clear	Download ss
Greedy BFS	28	28	231.40	35	500	euclidean	Low	Clear	Download ss
Greedy BFS	48	433	3369.60	35	500	manhattan	Low	Clear	Download ss
Greedy BFS	28	28	248.20	35	500	chebyshev	Low	Clear	Download ss
A* - Astar	28	293	2035.70	35	500	euclidean	Low	Clear	Download ss
A* - Astar	30	564	4585.50	35	500	hamming	Low	Clear	Download ss
A* - Astar	25	172	1564.10	50	500	minkowski5	Low	Clear	Download ss

Download CSV Clear all

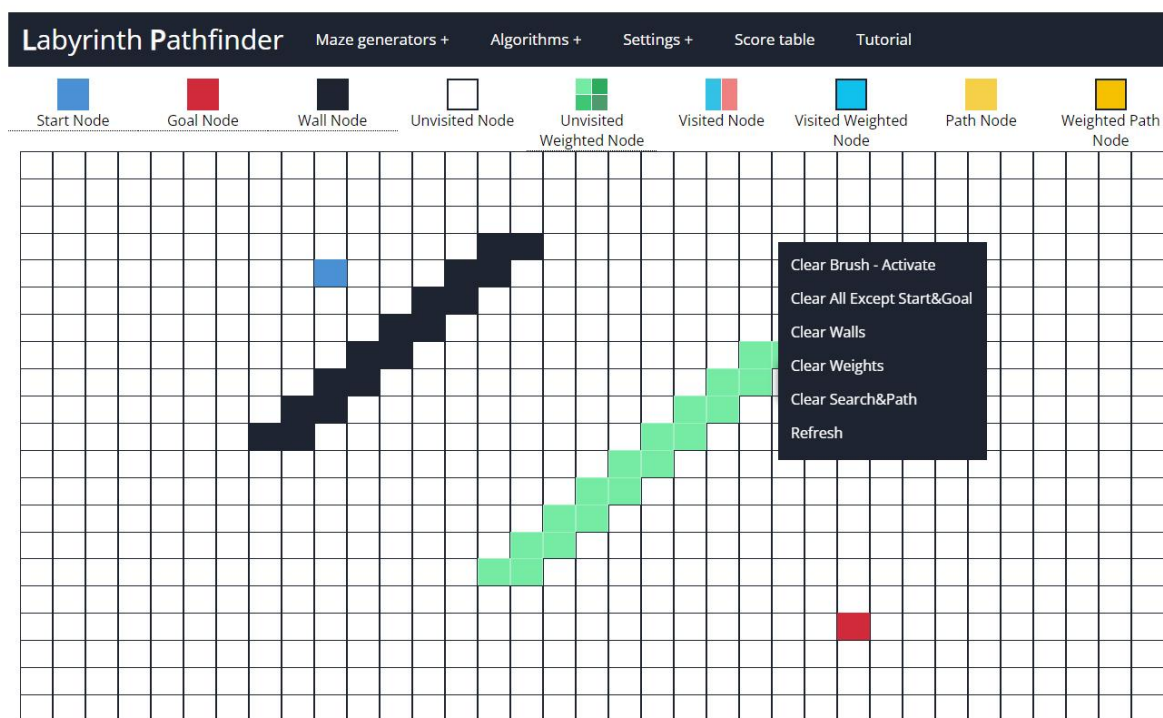
Слика 5.3 – Приказ табеле резултата

5.2.2 ТЕЛО

Приликом учитавања апликације, у простору стања се приказују почетни (енгл. *Start Node*) и циљани чвор (енгл. *Goal Node*) на случајним позицијама. Притиском левог тастера миша на позицију почетног или циљаног чвора, а потом притиском левог тастера миша на неку другу позицију омогућава се промена позиције чвора. Комбинацијом тастера *shift* и левог тастера миша се генеришу препреке на површини (енгл. *Wall Node*), док се комбинацијом тастера *alt* и левог тастера миша мења тежина чвора преко кога се прелази (енгл. *Weighted Node*). За тежину чвора се поставља она вредност која је унета у одељку за подешавања. Притиском десног тастера миша на чвор, чија је тежина већа од један, отвара се поље у које се може унети нова вредност за тежину (енгл. *Weight*). Уколико се исти поступак примени на било који други чвор, појављује се мени који пружа опције: четка за брисање (енгл. *Clear Brush*), брисање целе површине осим почетног и циљаног чвора (енгл. *Clear All Except Start&Goal*), брисање свих препрека (енгл. *Clear Walls*), брисање свих тежина већих од један (енгл. *Clear Weights*), брисање резултата претраге (енгл. *Clear Search&Path*) и освежавање површине (енгл. *Refresh*). На слици 5.5 и слици 5.6 је приказ наведених могућности.



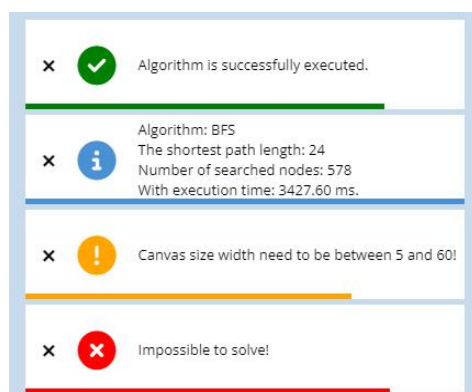
Слика 5.5 – Опција промене тежине чвора



Слика 5.6 – Додатне опције

Поред споменутих опција апликација подржава систем обавештења корисника о статусу претраге и детаљима претраге. Конкретно постоје четири врсте обавештења, а то су: успешно, додатна информација, упозорење и грешка. На слици 5.7 се може видети пример за свако обавештење. Дефинисан је временски период после којег ће обавештења бити уклоњена са површине. За

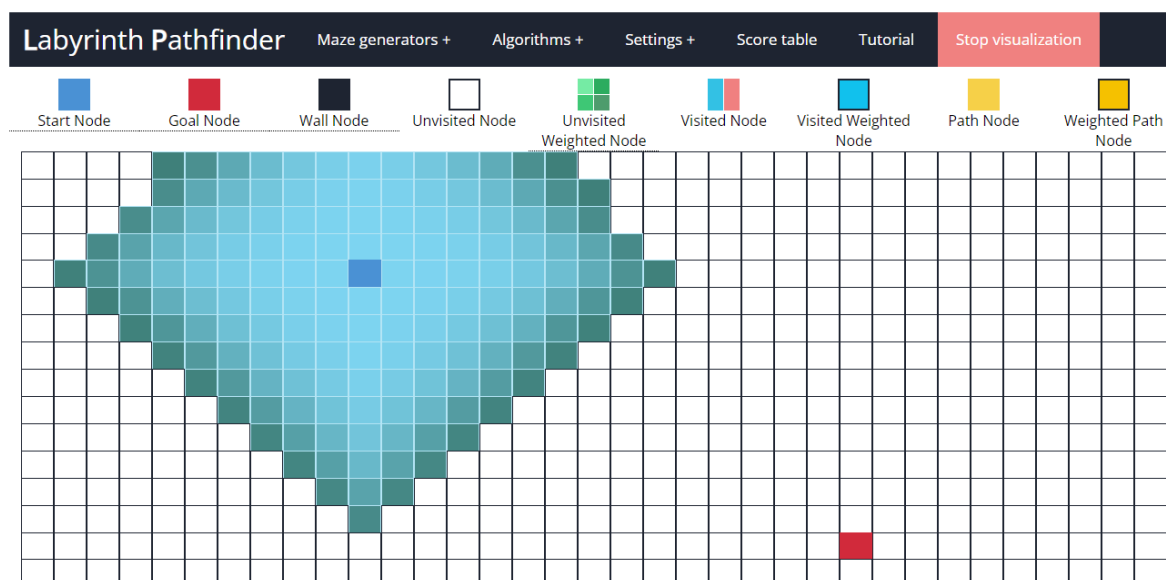
успешну претрагу и грешку је период од пет секунди, за упозорење је десет секунди и додатна информација нема временско ограничење. Свако обавештење може бити уклоњено и пре истека временског ограничења.



Слика 5.7 – Изглед обавештења

5.2.2.1 ПОКРЕТАЊЕ ПРЕТРАГЕ

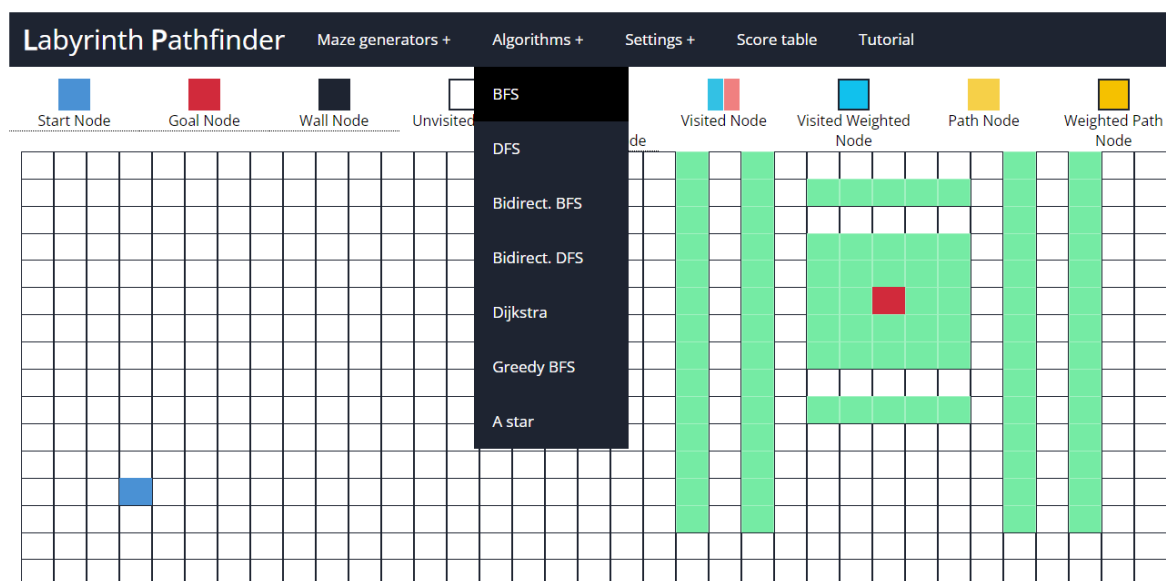
Након што је припремљена површина, корисник може изабрати претрагу коју жели визуелно да прикаже. Приликом покретања у заглављу ће се, поред наведених опција у *секцији 5.2.1*, појавити опција која омогућава заустављање претраге (енгл. *Stop visualization*). Пример се може видети на *слици 5.8*.



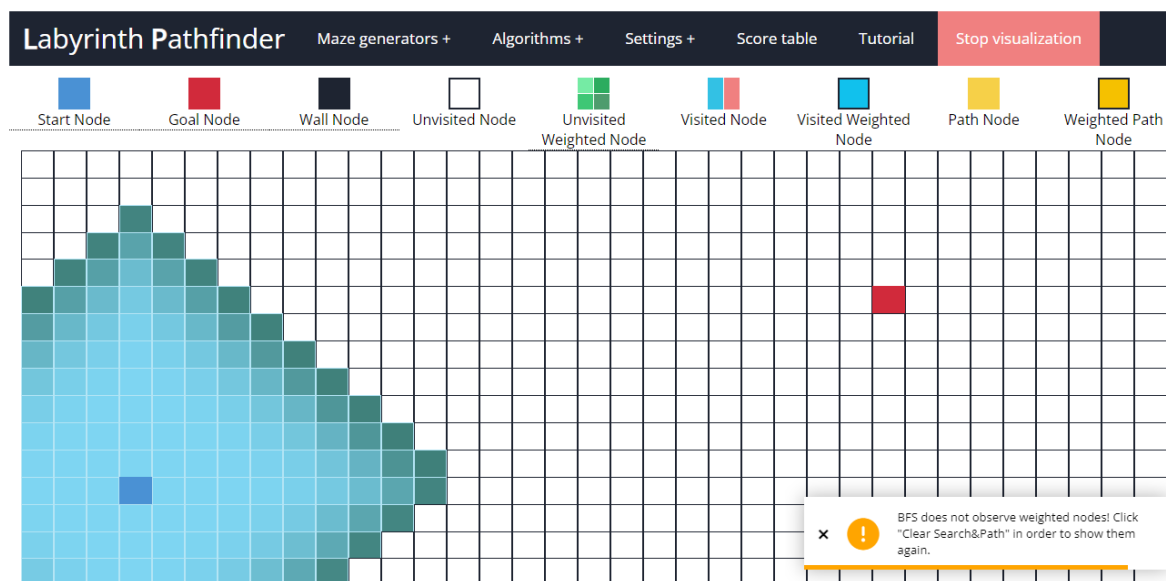
Слика 5.8 – Визуелизација претраге у ширину

Уколико је изабрана претрага која не узима у обзир тежине чворова, а на површини су дефинисани одређени чворови са тежином различитом од један, апликација ће привремено уклонити тежине тих чворова и поставити их на један. Поред тога, појавиће се обавештење о томе како претрага не подржава

такве чворове. Сценарио је приказан на *слици 5.9* и *слици 5.10*. Како би се вредности тежина поставиле на претходно уклоњене вредности, потребно је зауставити претрагу избором опције *Stop Visualization* или обрисати резултате претраге употребом помоћне опције *Clear Search&Path*.

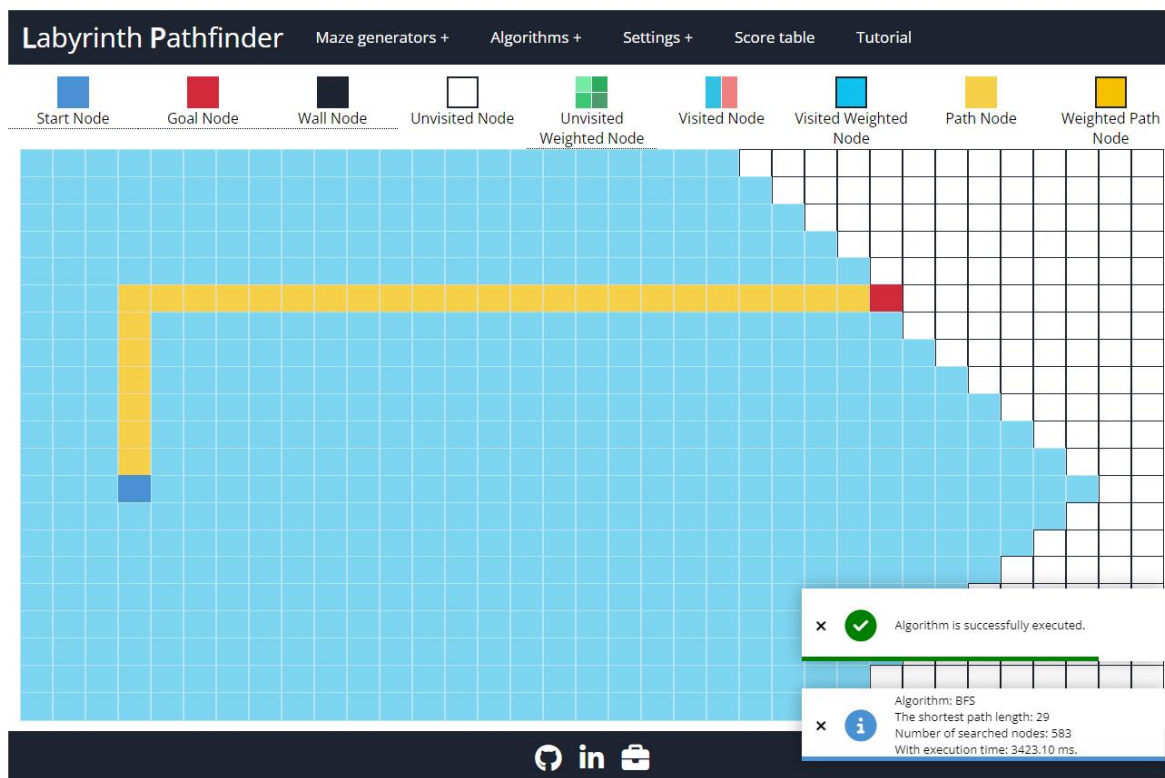


Слика 5.9 – Избор претраге која не узима у обзир тежину чворова



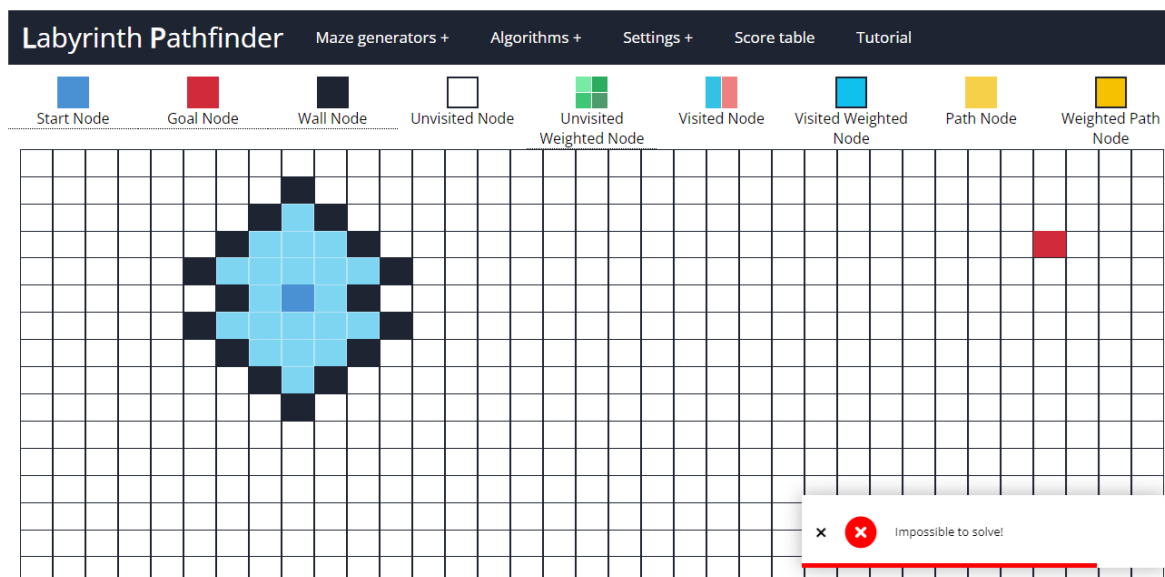
Слика 5.10 – Упозорење о претрази која не узима у обзир тежину чворова

Када се претрага заврши успешно, уклања се опција за прекид претраге и приказују се следећи елементи: најкраћи пут од почетног до циљаног чвора и два обавештења, једно да је претрага успешно завршена и друго које приказује сажетак информација о самој претрази. У позадини се чува слика екрана како би сценарио могао бити преузет из табеле резултата. Наведени елементи се могу видети на *слици 5.11*.



Слика 5.11 – Сценарио у којем је претрага успешна

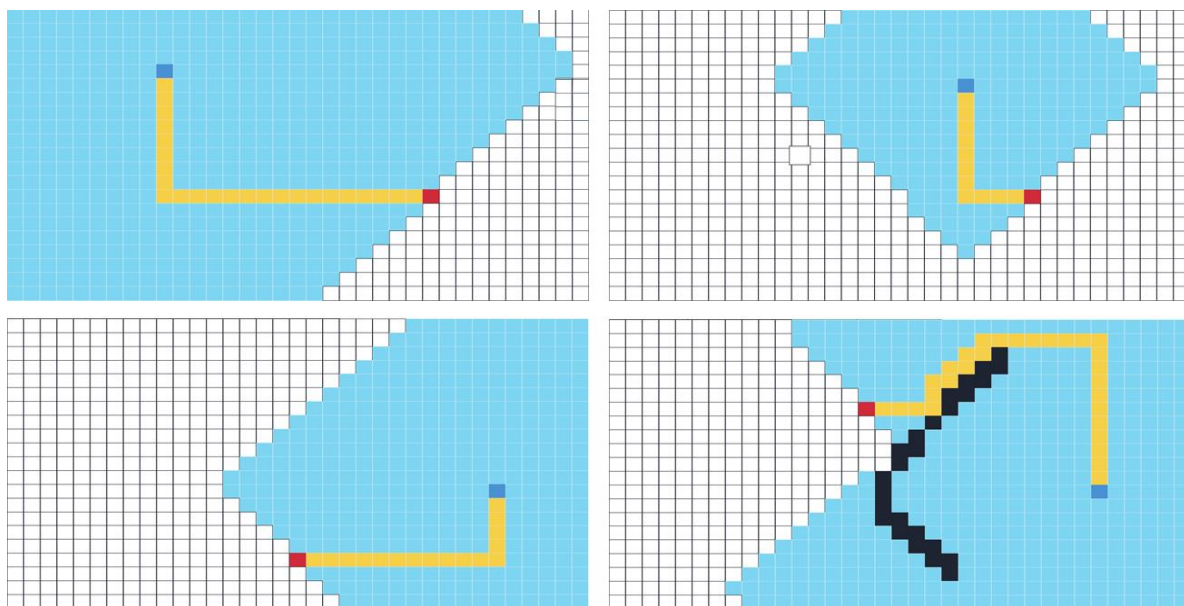
Уколико је претрагу немогуће довршити, на површини се појављује обавештење да није могуће извршити претрагу у тренутно постављеном окружењу. Сценарио је приказан на слици 5.12.



Слика 5.12 – Приказ немогућег сценарија

Завршетком претраге кориснику се омогућава симулација у реалном времену. То значи да ће корисник без поновног покретања претраге, већ само

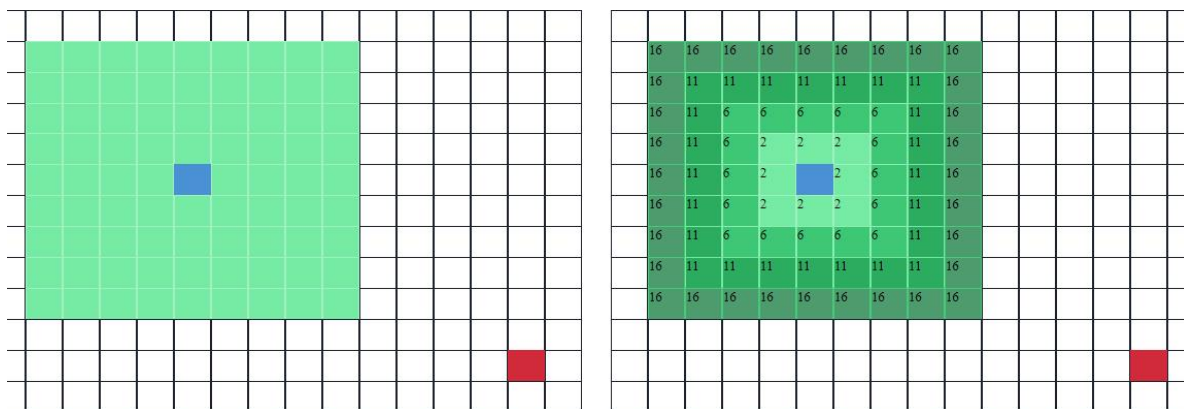
померањем почетног или циљаног чвора моћи да види резултат претраге. Ова симулација узима у обзир накнадне промене као што су додавање препрека и нових тежина за чворове, што се може видети на *слици 5.13*.



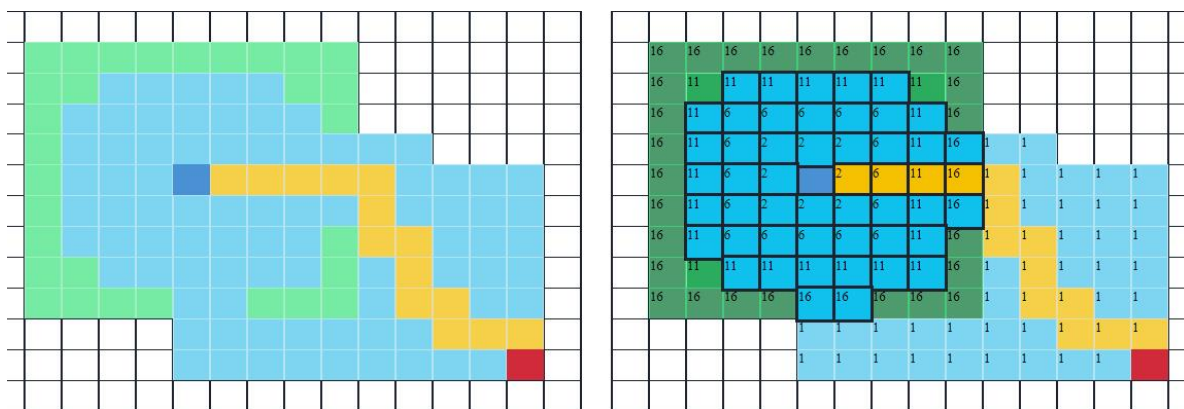
Слика 5.13 – Симулација у реалном времену претраге у ширину

5.2.2.2 НИВО ДЕТАЉА

Поменуто је у *секцији 5.2.1* да апликација нуди два нивоа детаља: низак ниво детаља (енгл. *Low*) и висок ниво детаља (енгл. *High*), који су приказани на *слици 5.14*. Мана ниског нивоа детаља је та што долази до губитка информације током претраге, конкретно у случају чворова са тежином што је и приказано на *слици 5.15*. Проблем је у томе што се не може уочити које чворове са тежином је претрага обухватила, а које не. Са друге стране, приликом употребе ниског нивоа детаља, перформансе претраге су боље из логичног разлога јер се мање података приказује на површини.



Слика 5.14 – Приказ ниског нивоа детаља (лево) и приказ високог нивоа детаља (десно)



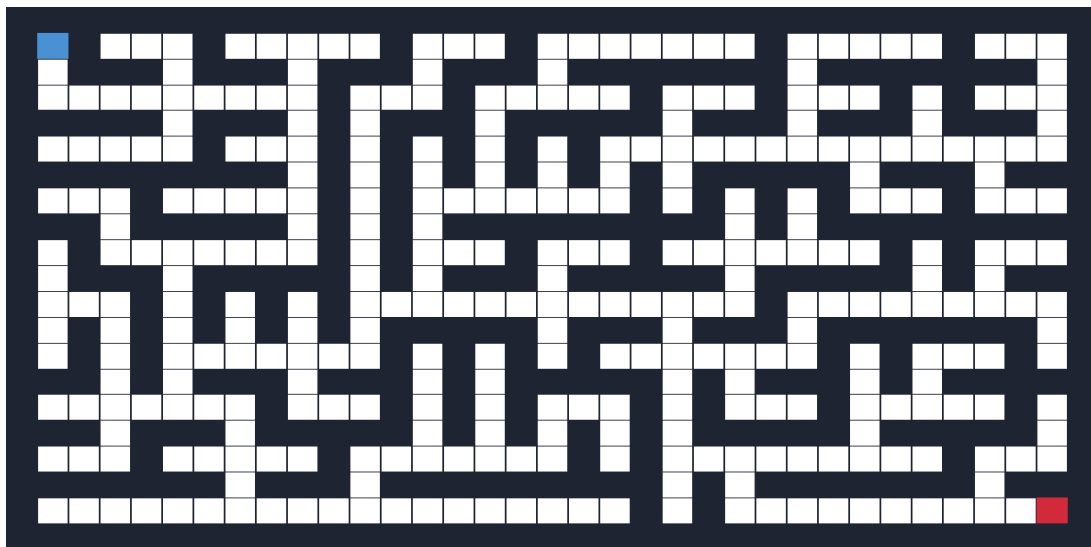
Слика 5.15 – Приказ ниског нивоа детаља (лево) и приказ високог нивоа детаља (десно)

5.3 ДЕМОНСТРАЦИЈА МОГУЋНОСТИ АПЛИКАЦИЈЕ КРОЗ ПРИМЕРЕ

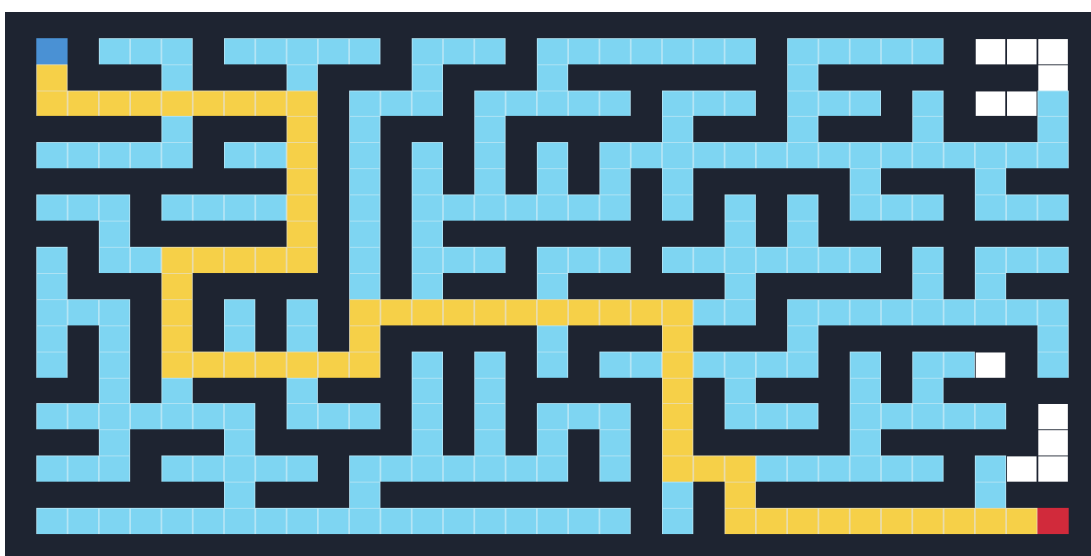
У наставку ће одређени примери бити демонстрирани кроз рад апликације. Неки од аспеката који ће бити покривени су: различите хеуристике примењене за исти алгоритам, како тежине утичу на развој претрага, разне препреке итд.

5.3.1 ПРИМЕР 1

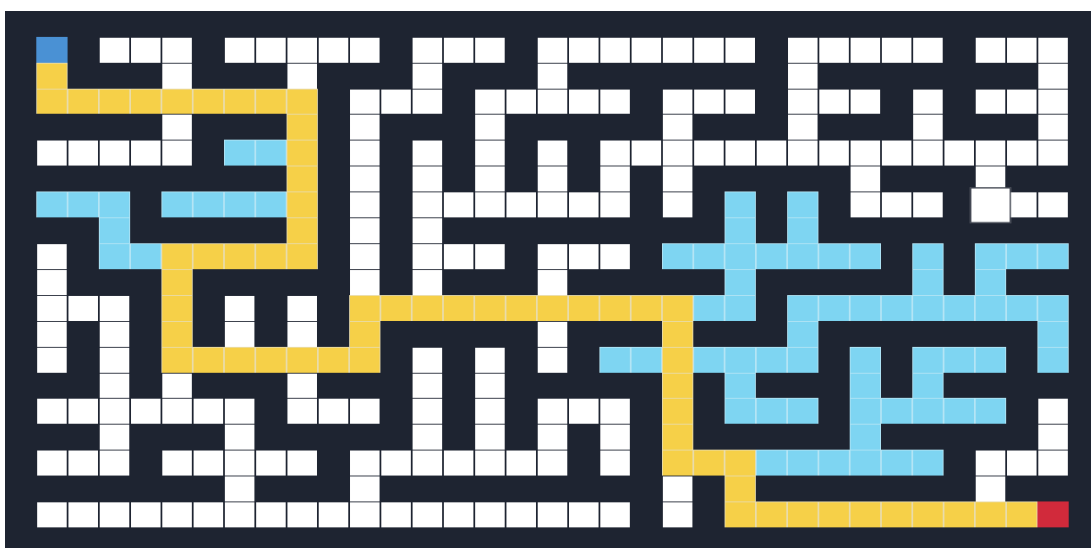
Окружење приказано на *слици 5.16* је формирано помоћу случајног Прим алгоритма. Покренут је процес визуелизације за све претраге. Резултат претраге у ширину је приказан на *слици 5.17*, резултат претраге у дубину је приказан на *слици 5.18*, резултат двосмерне претраге у ширину је приказан на *слици 5.19*, резултат двосмерне претраге у дубину је приказан на *слици 5.20*, резултат униформне претраге је приказан на *слици 5.21*, резултат похлепне претраге је приказан на *слици 5.22* и резултат А звезда претраге је приказан на *слици 5.23*.



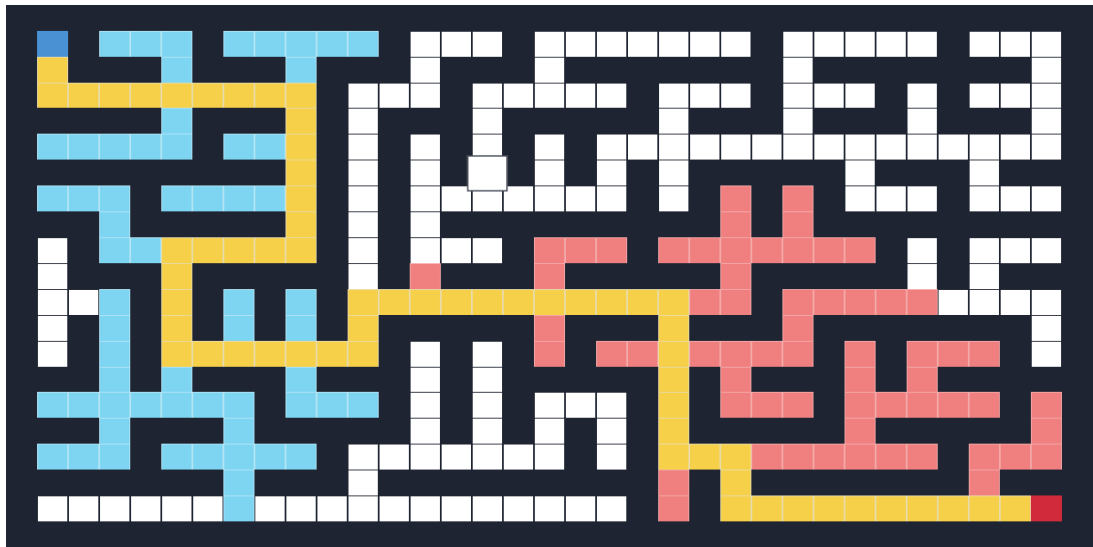
Слика 5.16 – Генерисани Прим лавиринт



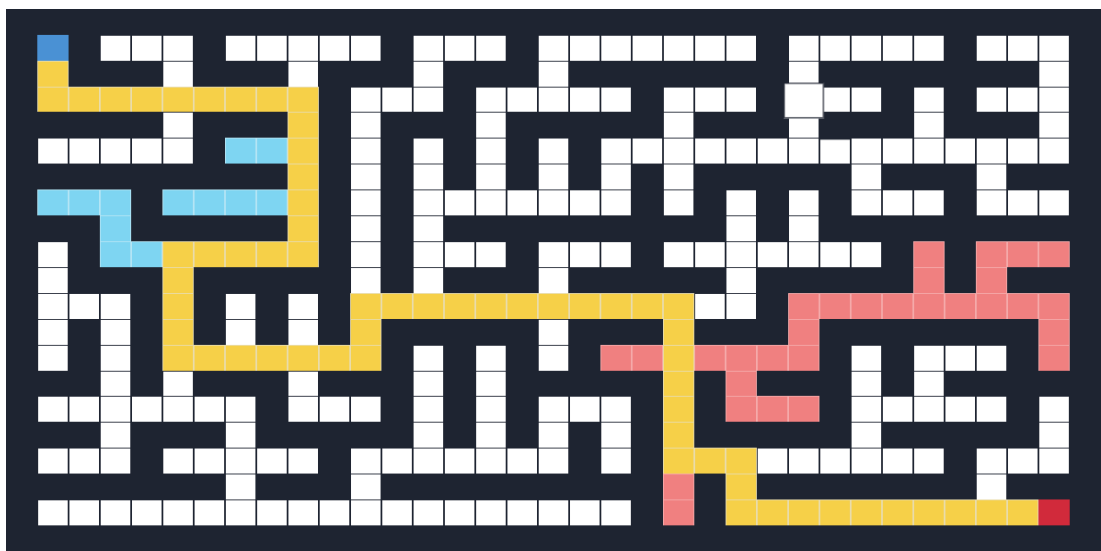
Слика 5.17 – Резултат визуелизације претраге у ширину



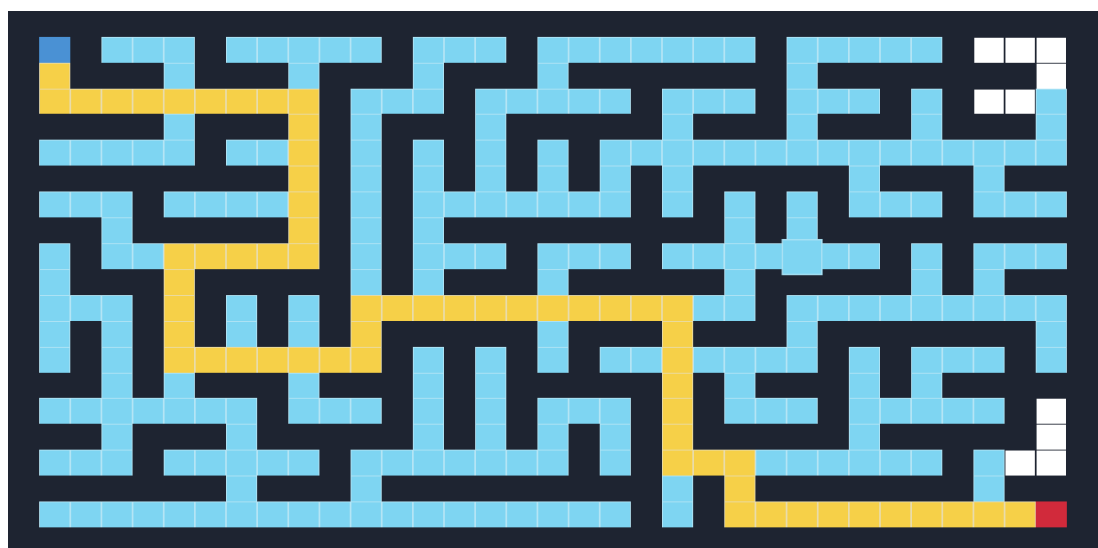
Слика 5.18 - Резултат визуелизације претраге у дубину



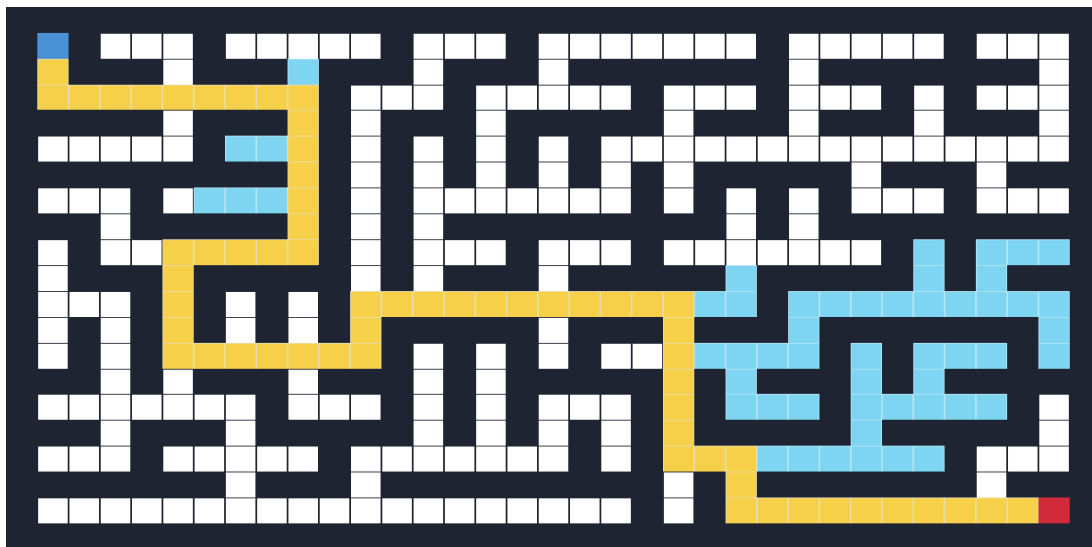
Слика 5.19 - Резултат визуелизације двосмерне претраге у ширину



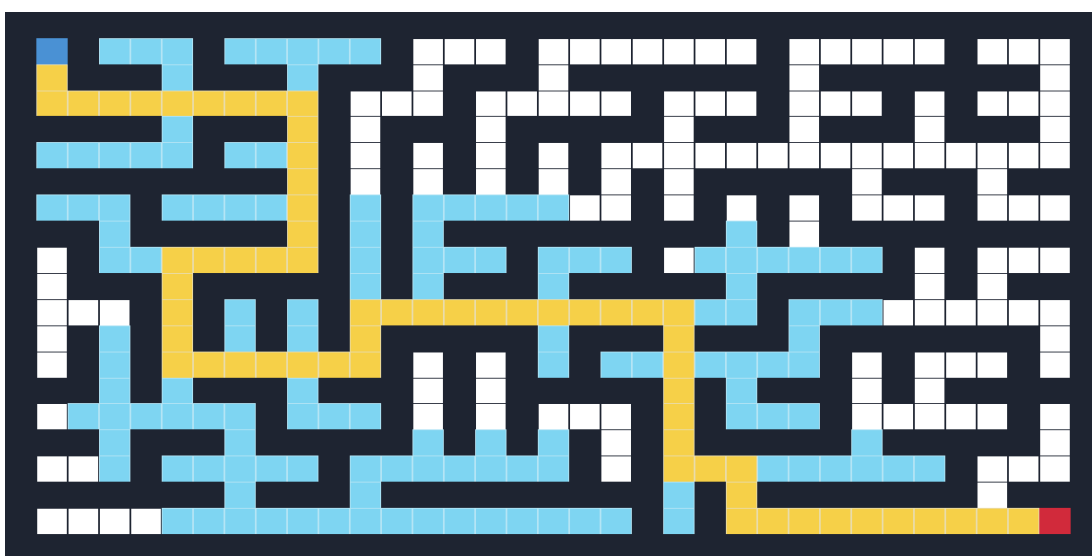
Слика 5.20 - Резултат визуелизације двосмерне претраге у дубину



Слика 5.21 - Резултат визуелизације униформне претраге



Слика 5.22 - Резултат визуелизације похлепне претраге



Слика 5.23 - Резултат визуелизације А звезда претраге

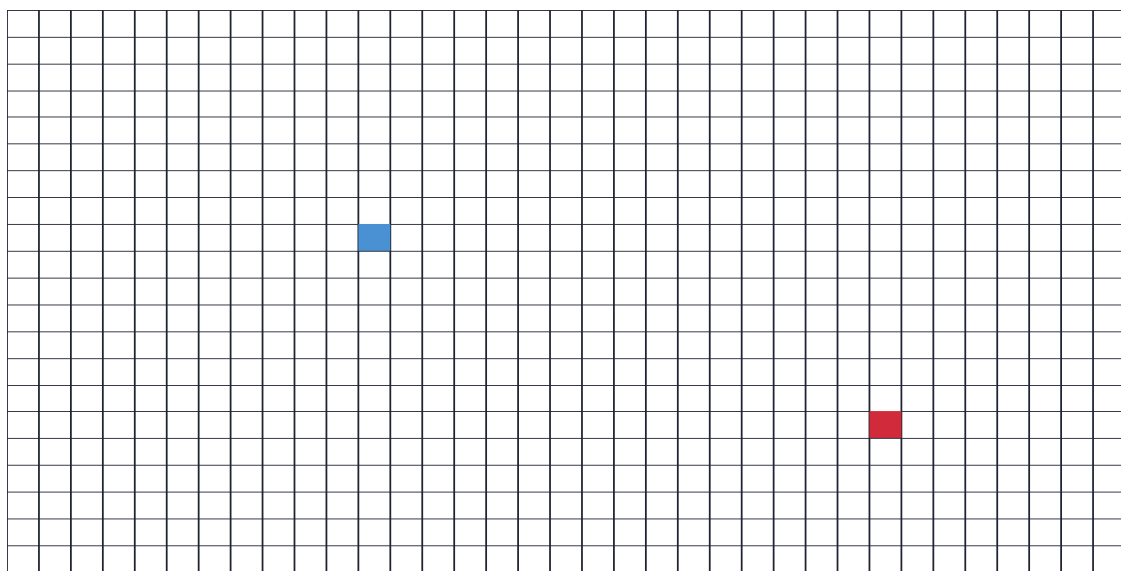
Перформансе за сваку претрагу су приказане на *слици 5.24*. Може бити примећено да све претраге дају пут исте дужине. Што се тиче брзине налаaska пута, двосмерна претрага у дубину дефинитивно даје најбољи резултат од само 297,60 милисекунди. Томе доприноси много мањи број посећених чворова током претраге.

Alg. name	Path length	Visited nodes	Time	Canvas	Speed	Heuristic	Details	Operations	
BFS	61	326	1872.90	35	500	/	Low	Clear	Download ss
DFS	61	133	741.60	35	500	/	Low	Clear	Download ss
Bidirect. BFS	61	184	514.70	35	500	/	Low	Clear	Download ss
Bidirect. DFS	61	103	297.60	35	500	/	Low	Clear	Download ss
Dijkstra	61	327	1854.10	35	500	/	Low	Clear	Download ss
Greedy BFS	61	114	769.80	35	500	euclidean	Low	Clear	Download ss
A* - Astar	61	195	1356.80	35	500	euclidean	Low	Clear	Download ss

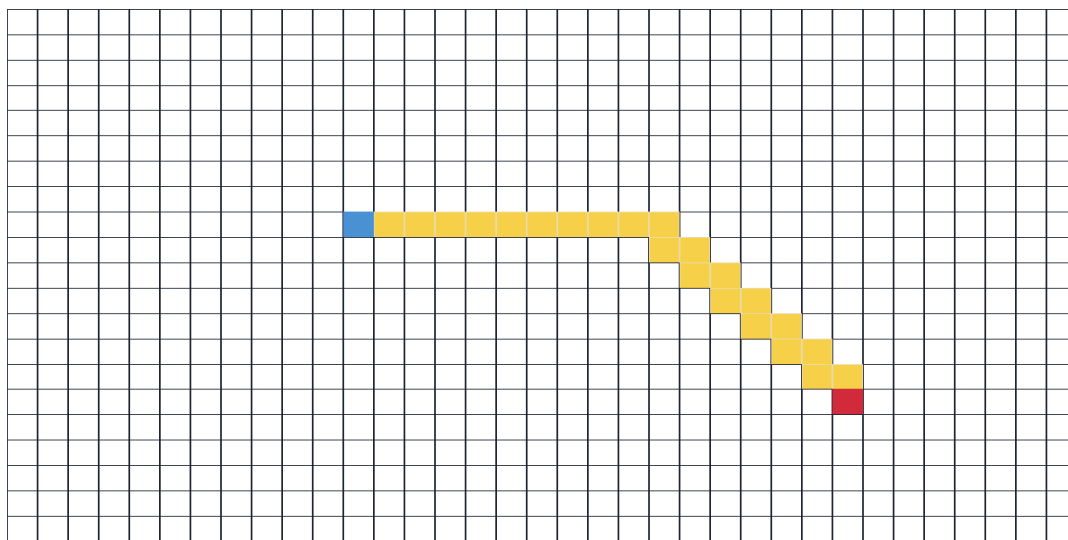
Слика 5.24 – Перформансе покренутих претрага

5.3.2 ПРИМЕР 2

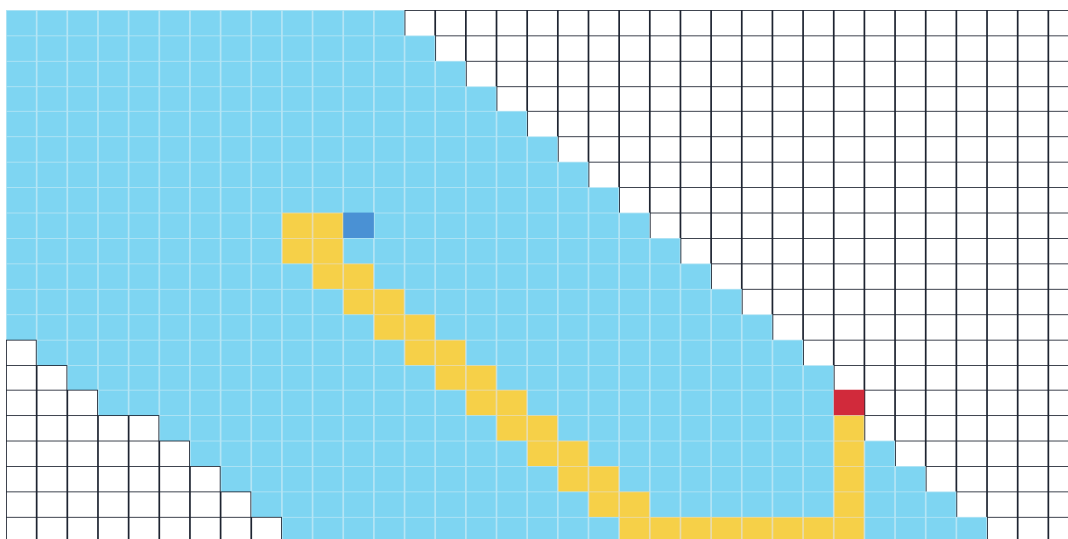
У следећем примеру се обраћа пажња на чињеницу како хеуристика утиче на исход претраге. Од навођених претрага, у апликацији су подржане похлепна претрага и А звезда претрага. Конкретно ће бити коришћена похлепна претрага и сценарио за развој претраге је приказан на *слици 5.25*. Резултат коришћења еуклидског растојања је приказан на *слици 5.26*, резултат коришћења Менхетн растојања је приказан на *слици 5.27*, резултат коришћења Хеминговог растојања је приказан на *слици 5.28*, резултат коришћења растојања Минковског је приказан на *слици 5.29*, резултат коришћења косинусног растојања је приказан на *слици 5.30*, резултат коришћења Чебишевљевог растојања је приказан на *слици 5.31* и резултат коришћења хеуристике хаверсинуса је приказан на *слици 5.32*.



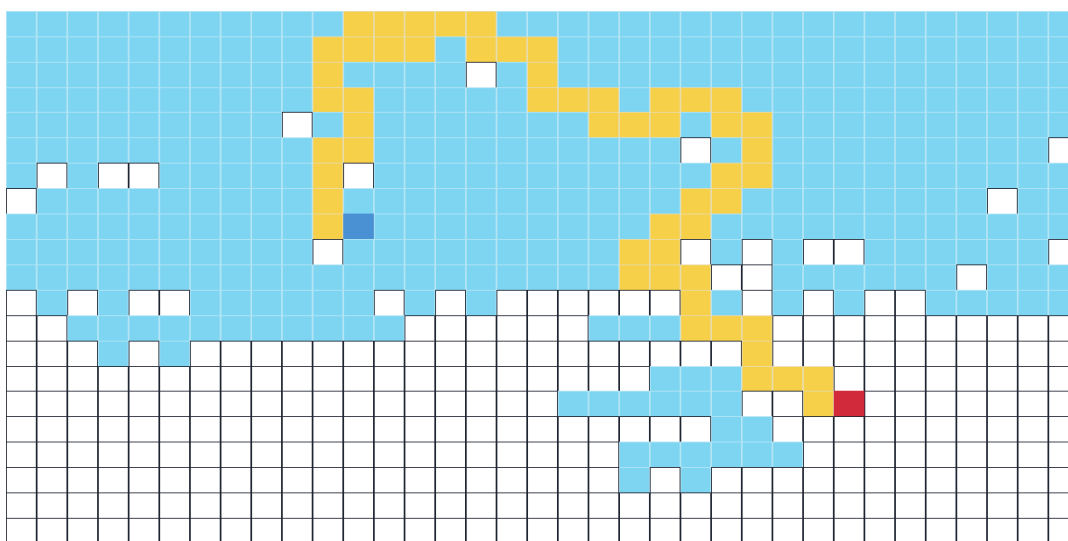
Слика 5.25 – Сценарио претраге



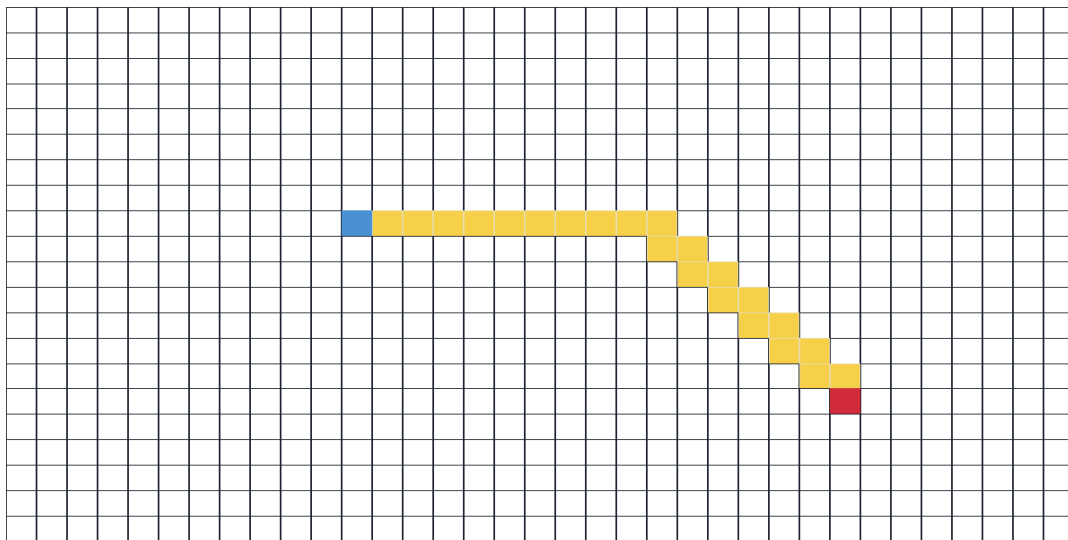
Слика 5.26 – Еуклидско растојање у похлепној претрази



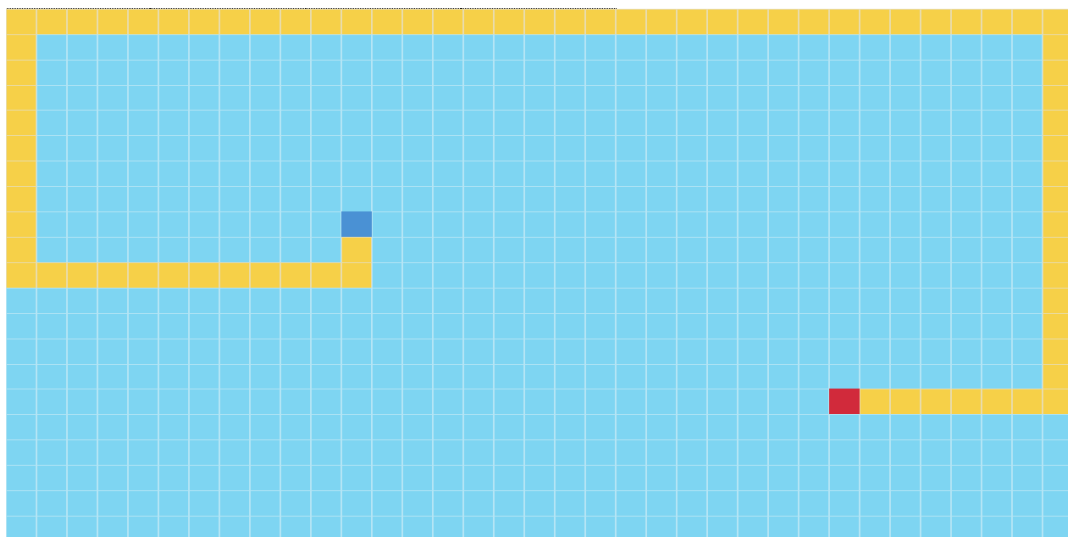
Слика 5.27 – Менхетн растојање у похлепној претрази



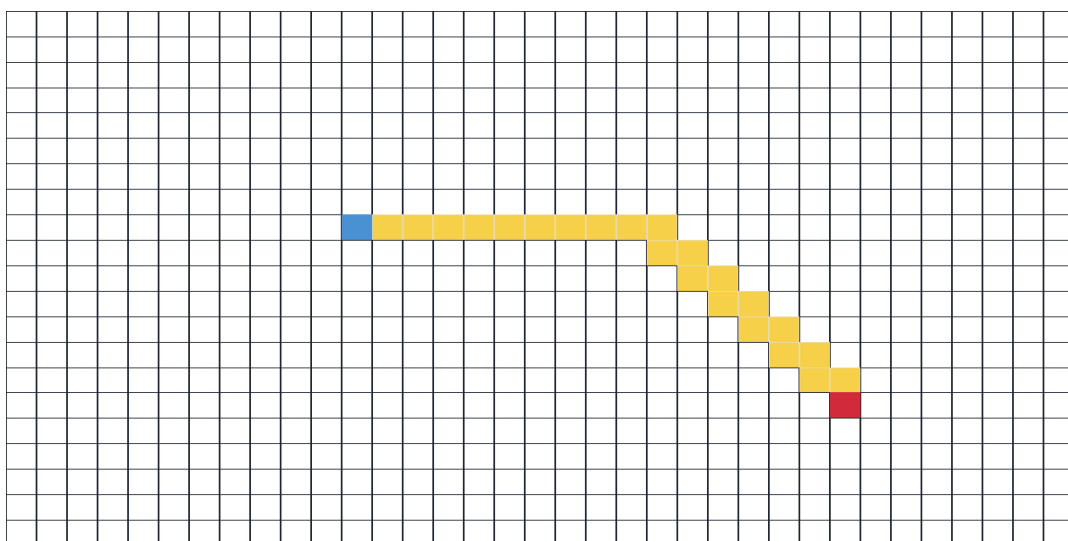
Слика 5.28 – Хемингово растојање у похлепној претрази



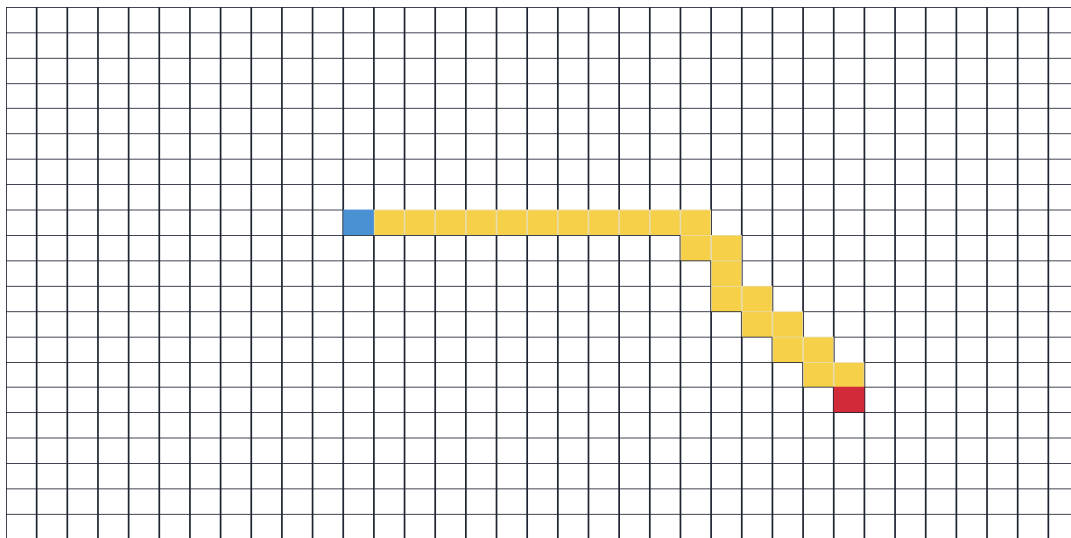
Слика 5.29 – Растојање Минковског са параметром 5 у похлепној претрази



Слика 5.30 – Косинусно растојање у похлепној претрази



Слика 5.31 – Чебишевљево растојање у похлепној претрази

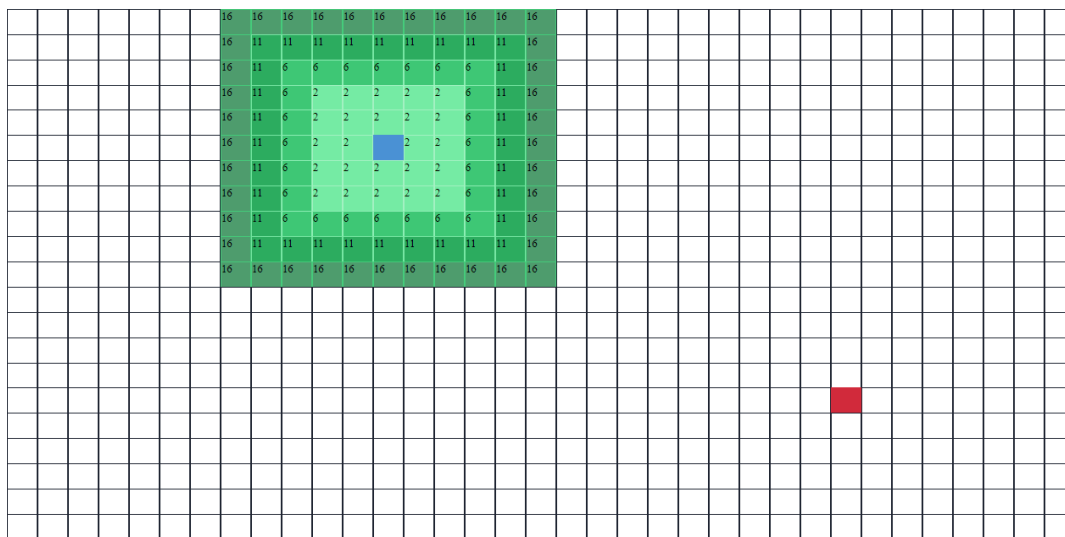


Перформансе похлепне претраге коришћењем сваке хеуристике засебно су приказане на *слици 5.33*. Приметно је да одабрана хеуристика има огроман утицај на исход претраге. Резултати из табеле доводе до закључка да уколико се за одређени навођени алгоритам одабере хеуристика која не одговара природи проблема, може доћи до знатног нарушавања перформанси претраге.

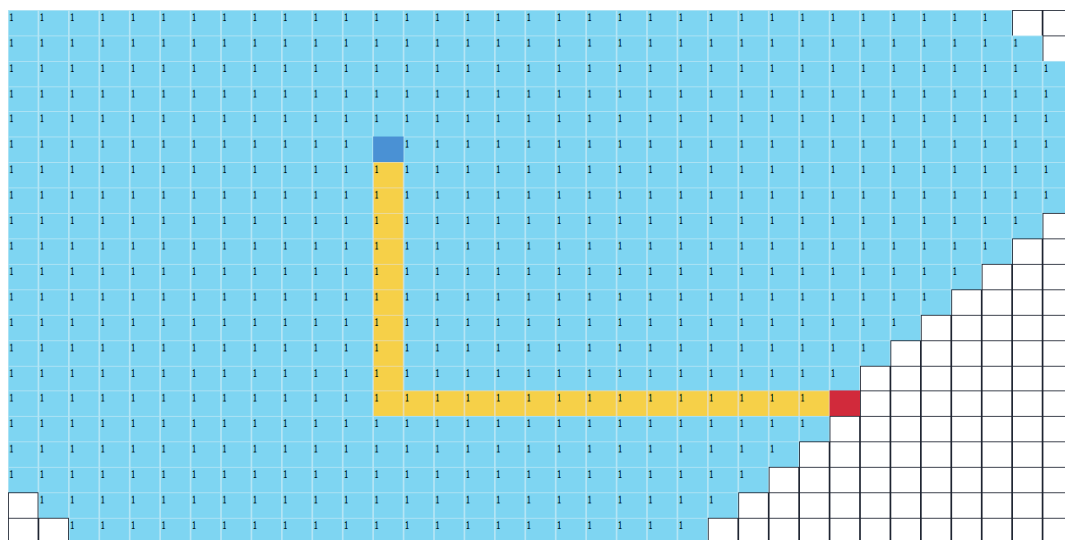
Alg. name	Path length	Visited nodes	Time	Canvas	Speed	Heuristic	Details	Operations	
Greedy BFS	22	22	195.10	35	500	euclidean	Low	Clear	Download ss
Greedy BFS	36	435	3534.10	35	500	manhattan	Low	Clear	Download ss
Greedy BFS	54	427	3969.30	35	500	hamming	Low	Clear	Download ss
Greedy BFS	22	22	203.60	35	500	minkowski5	Low	Clear	Download ss
Greedy BFS	78	733	6170.60	35	500	cosine	Low	Clear	Download ss
Greedy BFS	22	22	205.10	35	500	chebyshev	Low	Clear	Download ss
Greedy BFS	22	22	196.60	35	500	haversine	Low	Clear	Download ss

5.3.3 ПРИМЕР 3

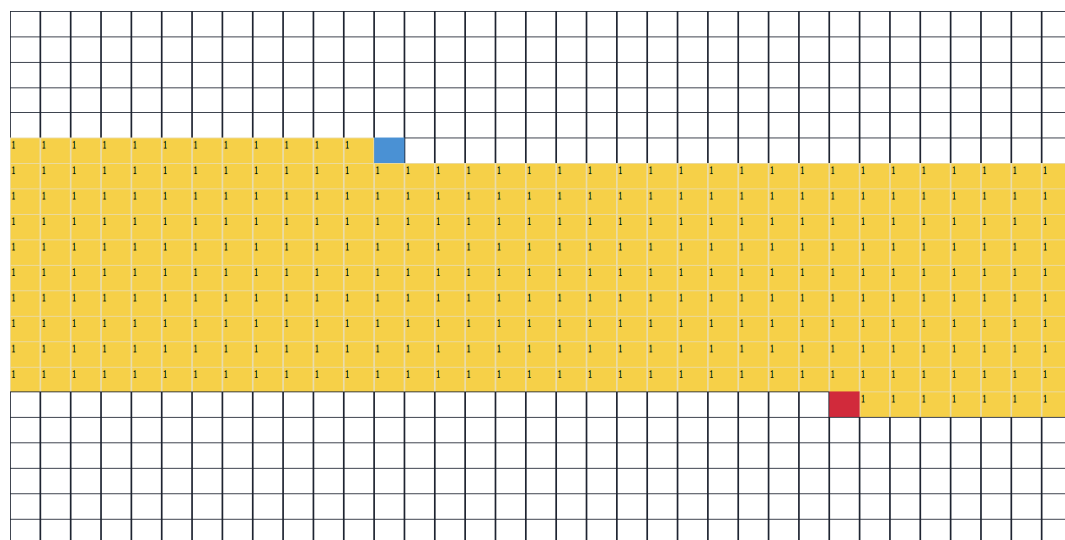
Последњи пример ће демонстрирати како апликација регулише ситуације у којима су примењени чворови са подешеним тежинама. Активиран је режим високих детаља како би се могли видети детаљи претраге. Сценарио је приказан на *слици 5.34*, резултат претраге у ширину је приказан на *слици 5.35*, резултат претраге у дубину је приказан на *слици 5.36*, резултат двосмерне претраге у ширину је приказан на *слици 5.37*, резултат двосмерне претраге у дубину је приказан на *слици 5.38*, резултат униформне претраге је приказан на *слици 5.39*, резултат похлепне претраге је приказан на *слици 5.40* и резултат А звезда претраге је приказан на *слици 5.41*.



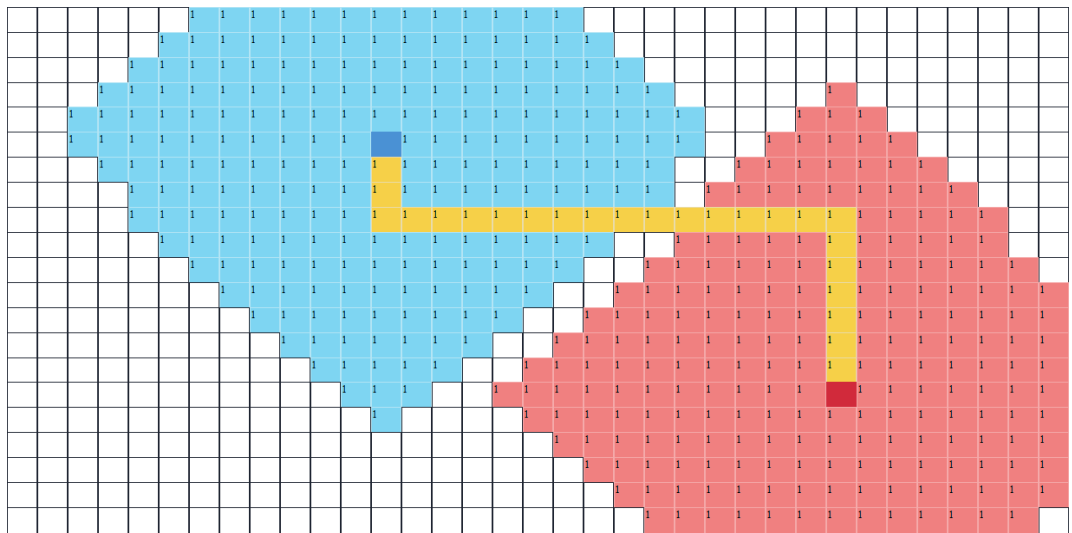
Слика 5.34 – Сценарио претраге



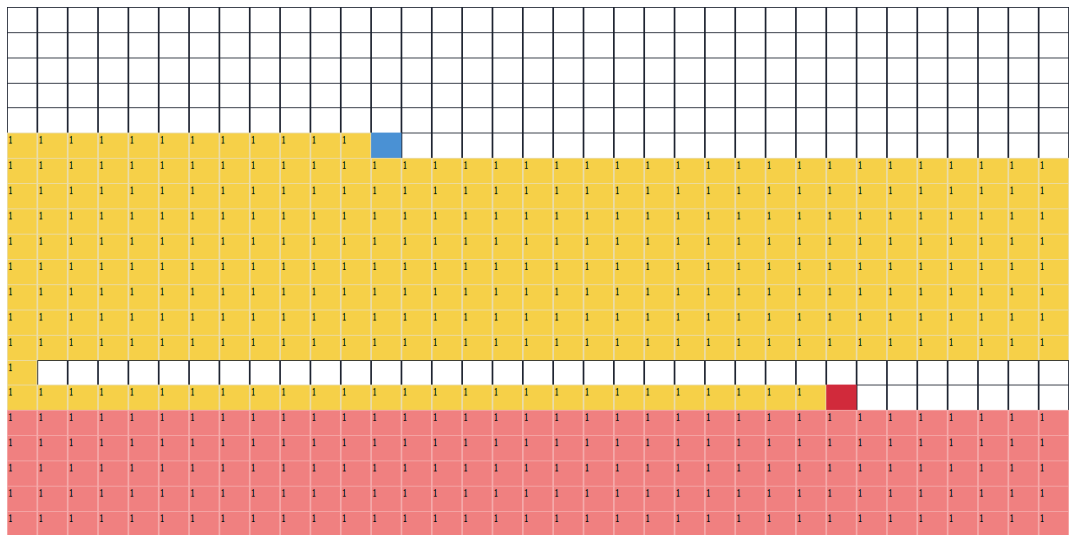
Слика 5.35 – Резултат претраге у ширину



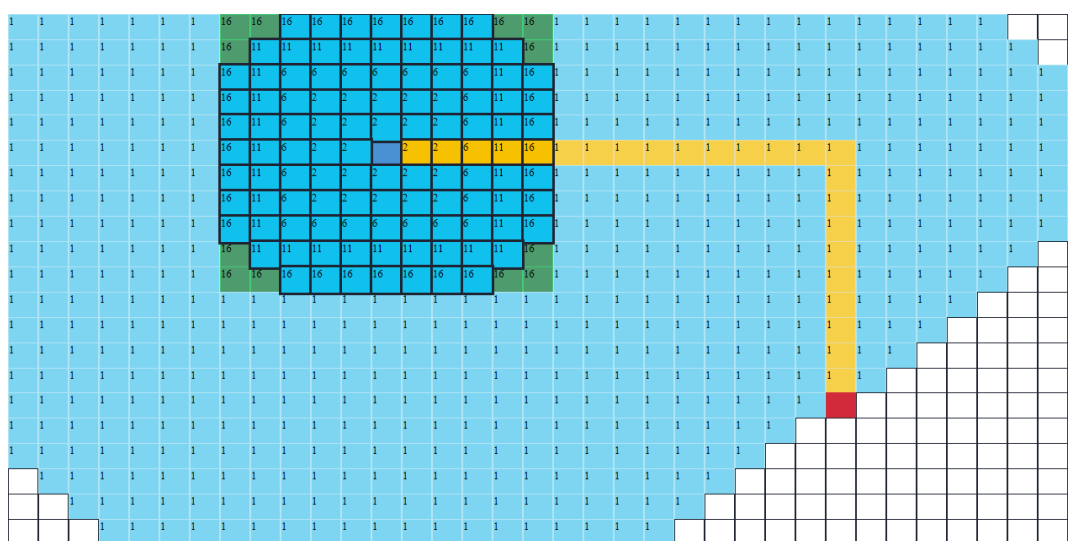
Слика 5.36 – Резултат претраге у дубину



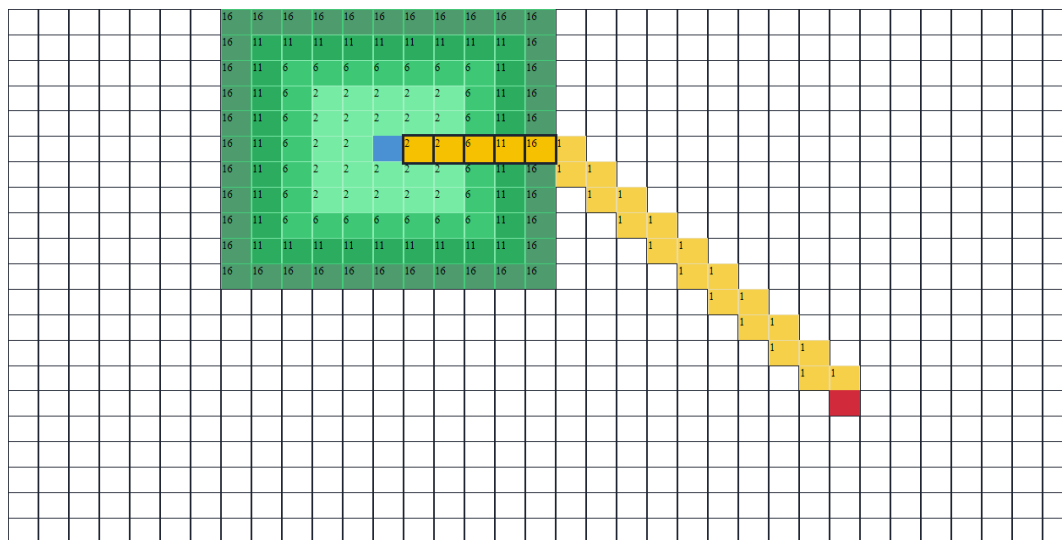
Слика 5.37 – Резултат двосмерне претраге у ширину



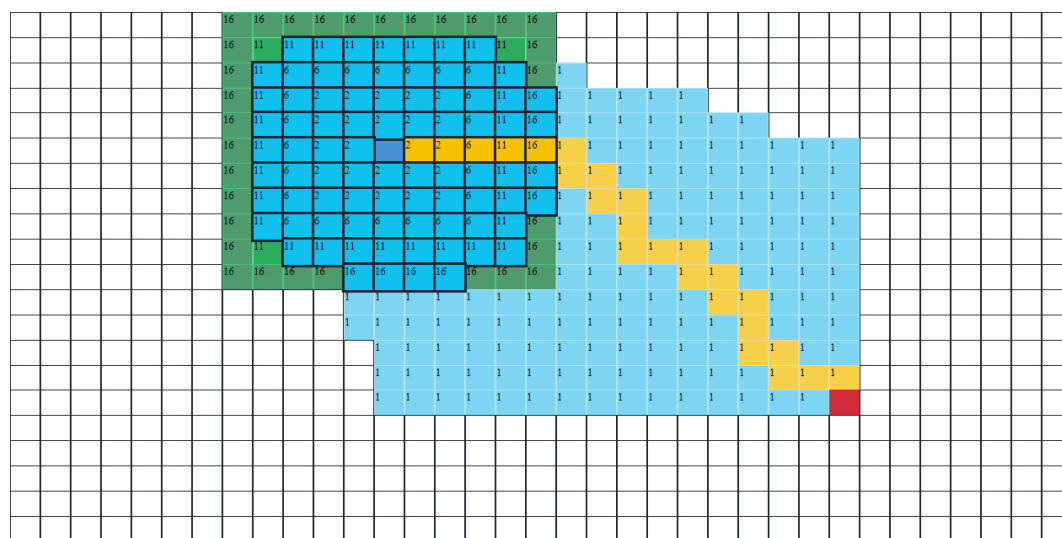
Слика 5.38 – Резултат двосмерне претраге у дубину



Слика 5.39 – Резултат униформне претраге



Слика 5.40 – Резултат похлепне претраге



Слика 5 – Резултат А звезда претраге

Објашњење које је дато у *секцији 5.2.2.1* је демонстрирано овим примером. Може се видети да иако постоје дефинисани чворови са тежинама вишим од један, једносмерне и двосмерне претраге у ширину и дубину их не узимају у обзир. На *слици 5.42* су приказане перформансе претраге.

Alg. name	Path length	Visited nodes	Time	Canvas	Speed	Heuristic	Details	Operations	
BFS	24	642	4141.10	35	500	/	Low	Clear	Download ss
DFS	334	334	1889.50	35	500	/	Low	Clear	Download ss
Bidirect. BFS	24	447	1749.20	35	500	/	Low	Clear	Download ss
Bidirect. DFS	320	495	2223.30	35	500	/	Low	Clear	Download ss
Dijkstra	24	521	5568.60	35	500	/	High	Clear	Download ss
Greedy BFS	24	19	254.80	35	500	euclidean	High	Clear	Download ss
A* - Astar	24	154	3982.30	35	500	euclidean	High	Clear	Download ss

Слика 5.42 – Резултати претраге

5.4 ПРЕДНОСТИ И НЕДОСТАЦИ АПЛИКАЦИЈЕ

У секцији 2.5 је донет закључак да апликације описане у секцији 2.1, секцији 2.2, секцији 2.3 и секцији 2.4, углавном имају слабу подршку за подешавање окружења, слаб утицај на тежине чворова и немогућност избора хеуристике. Апликација описана у овом раду, пружа опције којима превазилази наведене недостатке. Омогућава слободу у избору подешавања окружења; промену величине простора стања, промену брзине визуелизације, заустављање процеса визуелизације, омогућава избор хеуристика и врло угодне механизме за промену тежине чвора. Додатно, апликација нуди опцију режима детаља, могућност прегледа резултата претраге из табеле резултата, чување резултата у *png* формату и чување резултата у *csv* формату.

Као недостатак апликације може се издвојити: немогућност прилагођења корисничког интерфејса различитим димензијама екрана (енгл. *responsiveness*) и брзина визуелизације алгоритама претраге није оптимизована.

6. ЗАКЉУЧАК

У почетном делу рада су представљени и упоређени репрезентативни примери апликација које су послужиле као мотивација за израду овог софтвера. Након тога је уследио опис одабраних алгоритама претраге и дефинисање спецификације апликације. У последњем делу рада представљена је сама апликација са свим својим могућностима које пружа и примерима који их демонстрирају.

Апликација је формирана првенствено у едукативне сврхе. Покрива суштинске концепте који се уче на предметима из ове области. Из тог разлога се може користити као помоћни материјал. Међутим, постоје одређени аспекти у којима може бити унапређена. Неки од праваца у којима се даље може усавршавати апликација су:

- Додавање нових алгоритама претраге као што су ограничена претрага у дубину (енгл. *Depth-Limited Search*), итеративна претрага у дубину (енгл. *Iterative-Deepening Search*), претрага *IDA** (енгл. *Iterative deepening A* search*), претрага *SMA** (енгл. *Simplified Memory-Bounded A* Search*) и рекурзивна претрага по својству (енгл. *Recursive Best-First Search*). Већина поменутих претрага представља модификацију већ постојећих претрага.
- Додавање нових алгоритама за генерисање лавиринта као што су Крускалов (енгл. *Kruskal*), Боровкин (енгл. *Borůvka*), Вилсонов (енгл. *Wilson*), Алдус-Бродер (енгл. *Aldous-Broder*) итд.
- Омогућити кориснику већу контролу над самим процесом визуелизације. Пример може бити померање претраге унапред и уназад након што је иста завршена и на тај начин омогућити бољи увид у одређене детаље. Поред тога, била би погодна могућност промене простора стања у реалном времену док је процес визуелизације у току; то би подразумевало постављање нових и брисање постојећих препрека и промену тежина чворовима.
- Последња, али и најкрупнија промена јесте увођење тродимензионалног погледа на простор стања. Корисник би могао током претраге да бира поглед и угао из којег жели да прати визуелизацију.

7. БИОГРАФИЈА

Илија Ракчевић је рођен 23. 10. 1998. у Краљеву, где је стекао своје основно и средње образовање. Школске 2018/19 године уписује се на Факултет техничких наука на студијски програм Информациони инжењеринг. Положио је све испите предвиђене планом и програмом и стекао услов за одбрану завршног рада.

8. ЛИТЕРАТУРА

- [1] „Clement Mihailescu GitHub profile,“ github.com [Online], Доступно на: <https://github.com/clementmihailescu/Pathfinding-Visualizer>. [Приступано: 5. 3. 2023].
- [2] „Pathfinding-Visualizer,“ github.io [Online], Доступно на: <https://clementmihailescu.github.io/Pathfinding-Visualizer/>. [Приступано: 5. 3. 2023].
- [3] „Hisham Maged GitHub profile,“ github.com [Online], Доступно на: <https://github.com/hisham-maged10/path-finder>. [Приступано: 10. 3. 2023].
- [4] „Graph Algorithms Visualizer,“ github.io [Online], Доступно на: <https://hisham-maged10.github.io/path-finder/>. [Приступано: 10. 3. 2023].
- [5] „Angel Uriot GitHub profile,“ github.com [Online], Доступно на: https://github.com/angeluriot/Maze_solver. [Приступано: 10. 3. 2023].
- [6] „Maze solver,“ github.io [Online], Доступно на: https://angeluriot.com/maze_solver/. [Приступано: 10. 3. 2023].
- [7] „Seung Jae Yang GitHub profile,“ github.com [Online], Доступно на: <https://github.com/seungjae02/PathfindingVisualizer>. [Приступано: 10. 3. 2023].
- [8] Russel S, Norvig P. Artificial Intelligence: A Modern Approach. 3rd Edition. Pearson Education (Upper Saddle River, NJ, USA), pp. 78-95, 2010.
- [9] „Breadth First Search in Data Structure,“ techvidvan.com [Online], Доступно на: <https://techvidvan.com/tutorials/breadth-first-search/>. [Приступано: 10. 3. 2023].
- [10] „All You Need To Know About The Breadth First Search Algorithm,“ www.edureka.co [Online], Доступно на: <https://www.edureka.co/blog/breadth-first-search-algorithm/>. [Приступано: 10. 3. 2023].
- [11] „Artificial Intelligence – Uniform Cost Search (UCS),“ algorithmicthoughts.wordpress.com [Online], Доступно на: <https://algorithmicthoughts.wordpress.com/2012/12/15/artificial-intelligence-uniform-cost-searchucs/>. [Приступано: 11. 3. 2023].
- [12] „Depth First Search (DFS) in Data Structure,“ data-flair.training [Online], Доступно на: <https://data-flair.training/blogs/depth-first-search/>. [Приступано: 11. 3. 2023].
- [13] „Depth First Search (DFS),“ www.programiz.com [Online], Доступно на: <https://www.programiz.com/dsa/graph-dfs>. [Приступано: 11. 3. 2023].
- [14] „Balanced, bidirectional search,“ lavallo.pl [Online], Доступно на: <http://lavallo.pl/planning/node237.html>. [Приступано: 11. 3. 2023].
- [15] „Bidirectional Search?,“ www.tutorialspoint.com [Online], Доступно на: <https://www.tutorialspoint.com/bidirectional-search>. [Приступано: 11. 3. 2023].

- [16] „Explain Bidirectional Search,“ www.ques10.com [Online], Доступно на: <https://www.ques10.com/p/48043/explain-bidirectional-search-1/>. [Приступано: 11. 3. 2023].
- [17] „What is greedy-best first search?,“ ai-master.gitbooks.io [Online], Доступно на: <https://ai-master.gitbooks.io/heuristic-search/content/what-is-greedy-best-first-search.html>. [Приступано: 5. 3. 2023].
- [18] „On the Completeness of Best-First Search Variants That Use Random Exploration,“ ojs.aaai.org [Online], Доступно на: <https://ojs.aaai.org/index.php/AAAI/article/view/10081>. [Приступано: 14. 3. 2023].
- [19] „Understanding best-first search,“ indiaai.gov.in [Online], Доступно на: <https://indiaai.gov.in/article/understanding-best-first-search>. [Приступано: 11. 3. 2023].
- [20] „A* Search Algorithm,“ www.geeksforgeeks.org [Online], Доступно на: <https://www.geeksforgeeks.org/a-search-algorithm/>. [Приступано: 11. 3. 2023].
- [21] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT Press. pp. 634-641.
- [22] Mazes for Programmers by Jamis Buck. The Pragmatic Programmers, LLC. pp. 51-60.
- [23] „What is HTML? An Introduction,“ codeinstitute.net [Online], Доступно на: <https://codeinstitute.net/global/blog/what-is-html-and-why-should-i-learn-it/>. [Приступано: 14. 3. 2023].
- [24] „A Brief History of HTML,“ www.washington.edu [Online], Доступно на: https://www.washington.edu/accesscomputing/webd2/student/unit1/module3/html_history.html. [Приступано: 14. 3. 2023].
- [25] „A brief history of CSS until 2016,“ www.w3.org [Online], Доступно на: <https://www.w3.org/Style/CSS20/history.html>. [Приступано: 14. 3. 2023].
- [26] „A Brief JavaScript History,“ launchschool.com [Online], Доступно на: <https://launchschool.com/books/javascript/read/introduction>. [Приступано: 14. 3. 2023].
- [27] „The History of JavaScript: Everything You Need to Know,“ www.springboard.com [Online], Доступно на: <https://www.springboard.com/blog/data-science/history-of-javascript/>. [Приступано: 14. 3. 2023].